我的研发实践：高准确率AICoding工作流设计

交易业务技术团队 大淘宝技术 2025年10月27日 02:22 浙江 1077人



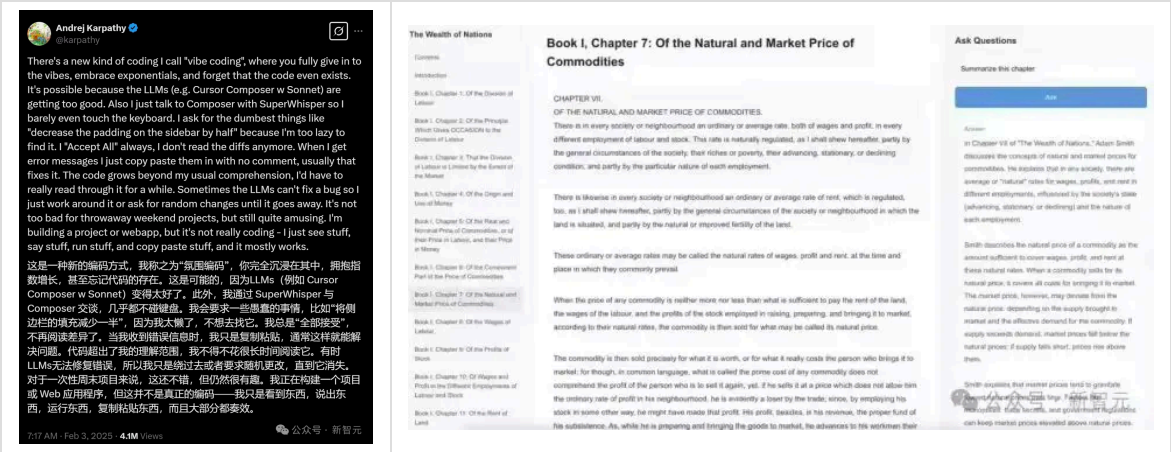
本文系统性地探讨了如何通过AI技术提升研发效能，尤其是在企业级复杂场景下的应用实践。文章以“氛围编程”（Vibe Coding）为切入点，指出尽管AI生成代码在效率上显著提升（代码生产率中位数提高17%），但在质量保障方面仍面临挑战。作者结合自身在交易团队的实践经验，提出应聚焦于高频、重复且熟悉的非业务需求场景（如AB实验下线、Switch开关治理）进行AI提效突破。

为此，团队设计了一套高准确率的AICoding工作流，融合MCP、A2A和AG-UI三大协议，构建了基于Single-Agent架构的智能生码系统。通过精细化提示词工程、动态上下文注入与标准化工作流编排，实现了AB实验推全/下线等任务的自动化生成与发布。文章重点强调：选择细分、可控的场景，结合业务深度知识，沉淀可复用的工作流模板，是实现安全、高效、可推广AI研发提效的关键路径。最终目标是在保证90%以上正确率的前提下，解放研发人力，聚焦高价值创新。

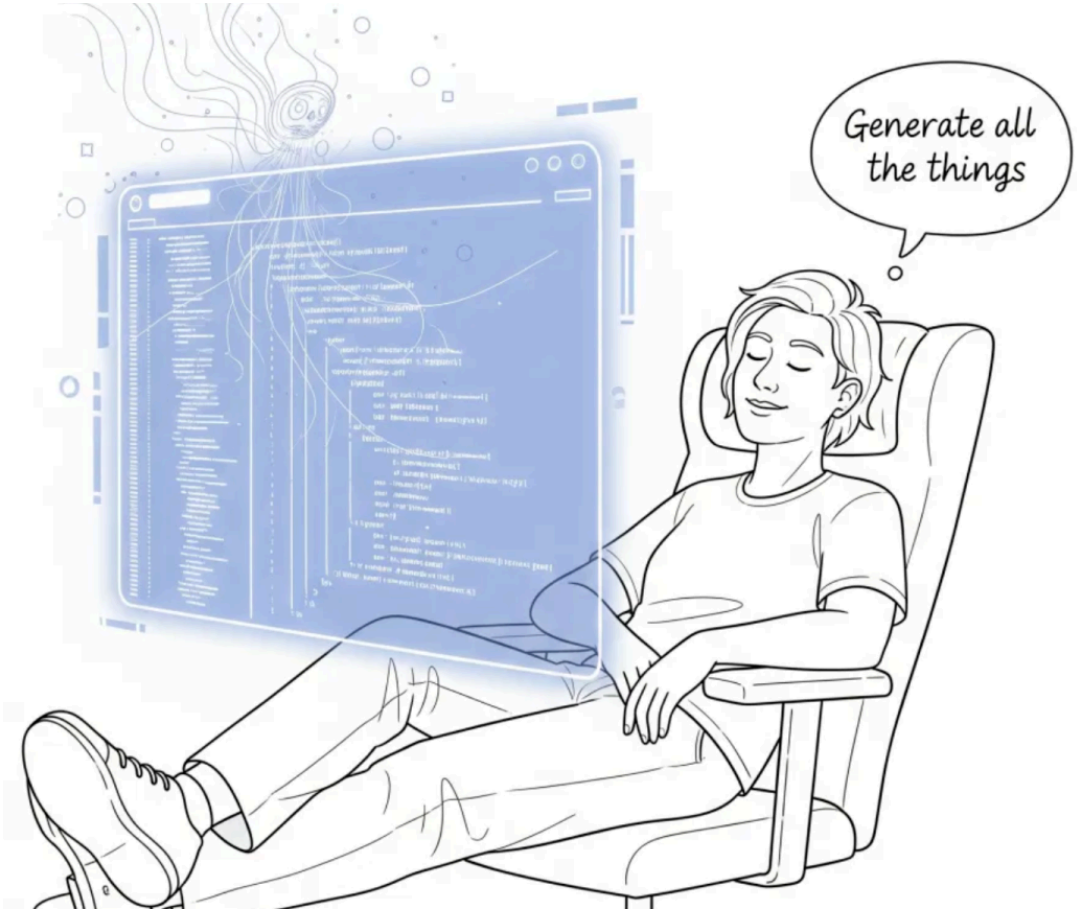


从氛围编程说起

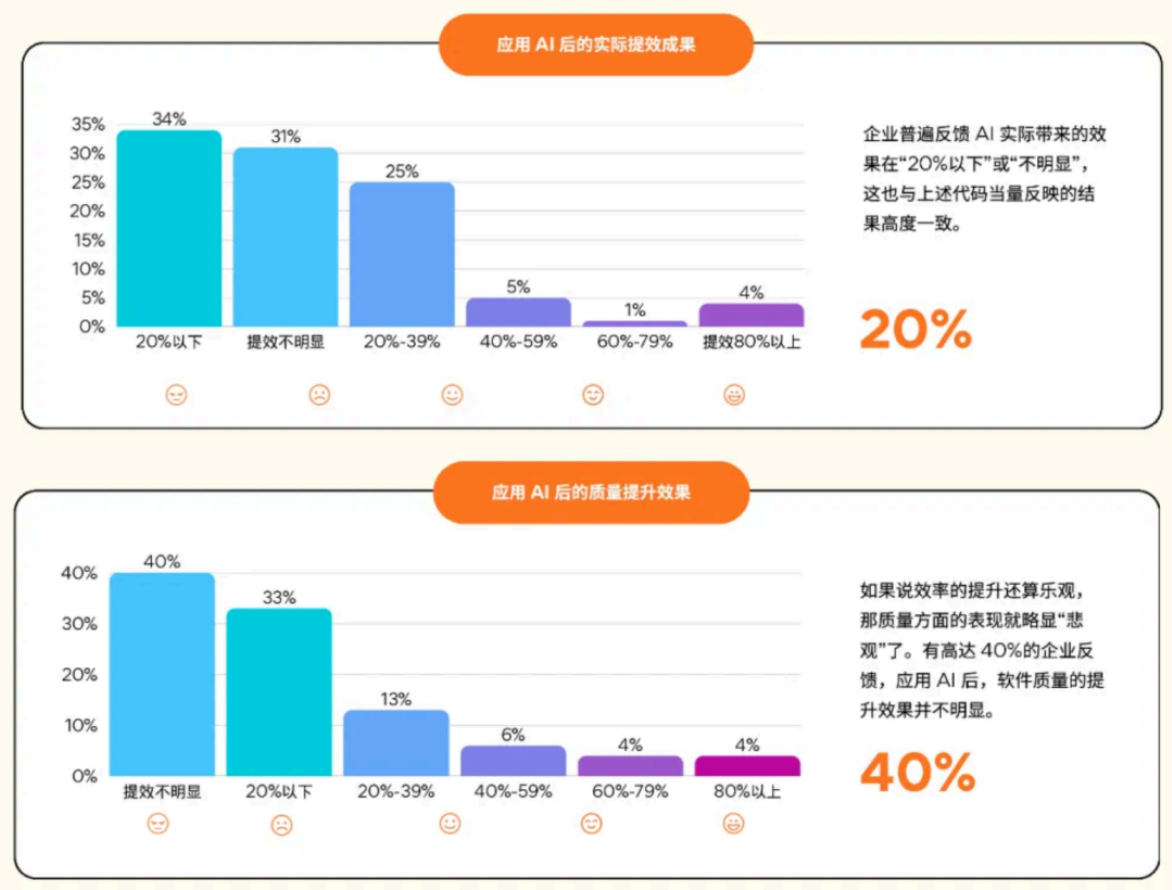
最近「Vibe Coding」（氛围编程）概念爆火！Y Combinator最新数据显示，其四分之一的W25届初创公司竟有95%的代码库由AI生成。Andrej Karpathy、吴恩达等AI大佬纷纷表示看好这种新趋势。Karpathy大神的话说就是，**「我只是看到东西，说出东西，运行东西，复制粘贴东西，而且大部分情况都奏效」**。



乍一看95%的代码生成，是不是以后程序员的代码都可以依靠AI进行生成，通过「Vibe Coding」这条路直接根据提升企业场景下的日常研发效率？同样我不禁思考「假设一家95%的代码都是AI生成的初创公司进入市场，一两年后，他们的产品拥有1亿用户。它会崩溃吗？」，但是怀疑之后我又清醒的认识到「**AI这不是一时潮流，它不会消失。氛围编程已经成为一种主要的编码方式。**如果我不这样做，可能就会被甩在后面」。



根据2025 年度《DevData 研发效能基准报告》指出：主观数据中部分企业反映应用 LLM 后效率有中高幅度提升，客观数据显示已落地应用 LLM 的企业代码生产率中位值较未应用企业有明显增幅，二者相互呼应，凸显 LLM 对代码产出效率提升的实际价值：



从上图可以看出：应用 AI 的团队效率确实有提升，**代码生产率中位数提升了 17%**。但在质量方面，约 **40%的企业反馈 AI“效果不明显”**，说明其在处理有复杂依赖和知识的质量保证工作时仍面临挑战。这部分反映的依然是LLM应用在企业领域上研发提效的实际场景。「Vibe Coding」氛围编程如火如荼，但是企业研发提效上效果和质量都还没有标准化的范式，在研发团队提效在全民AI时代，我们能做什么，能做到什么程度，在成本日益收紧的今天，如何驱动团队以更高效能交付更高质量的产品？

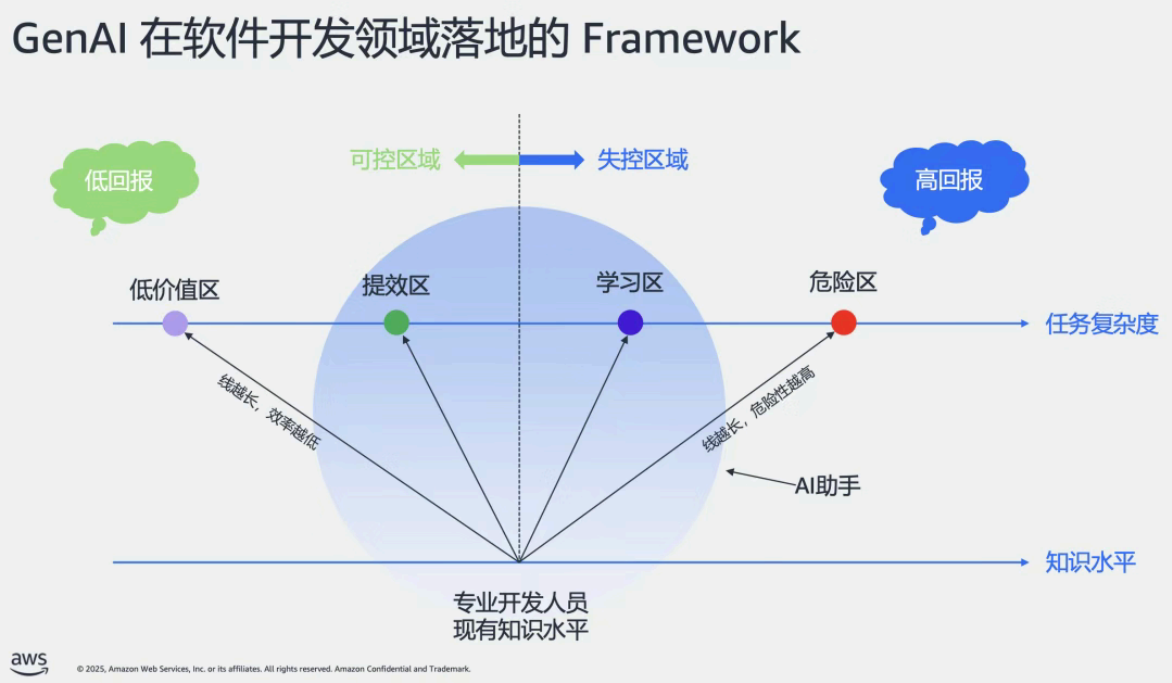


应该从哪里入手

就在今年6月，亚马逊研发技术中心的负责人及团队内一线的 AI 研发技术同学来我们公司，组织了一场《AI 如何助力研发提效？》的技术交流和分享，让我印象深刻的是其中提到：GenAI in 研发效能中为 **项目代码和依赖的JDK升级** 创建了一套独有提示词工程工作流，通过对JDK升级过程中的涉及到的修改内容、注意事项、最佳实践进行提炼总结和沉淀，在AWS这家人均需要0.3研发人力投入日常运维稳定性保障的公司，节约了数以千日的运维成本。



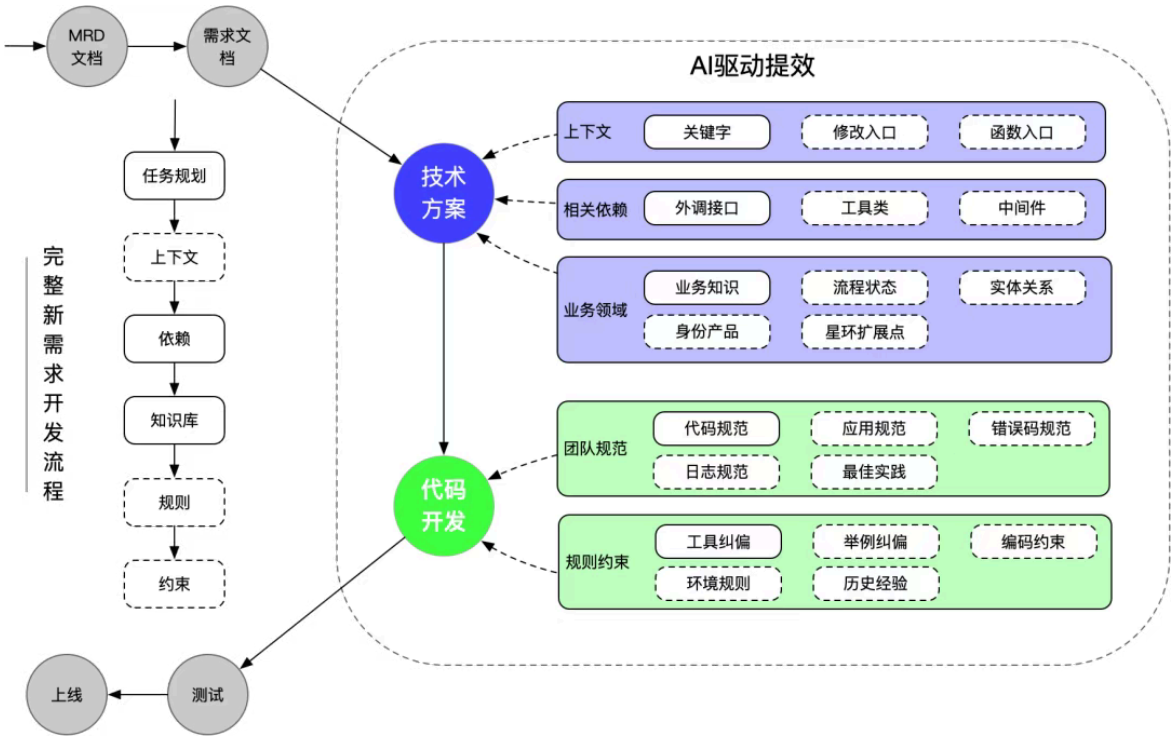
对于GenAI在软件领域落地的Framework不难看出，我们业务工程提效领域的突破点，**应该选在原来你做的就很好的领域，在可控的范文内去做提效**；同时沉淀经验模版，利用它去做学习，学习以前学不来的部分；我觉得AWS在选择JDK升级这个运维工程是非常熟悉的领域，沉淀工作流模版，完成不同应用场景下的适配，进行提效是非常不错的选题。



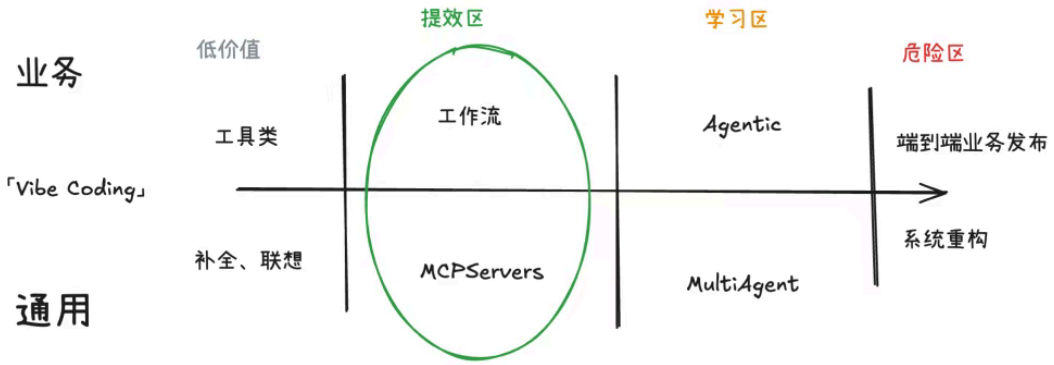
当时在现场的我就在想，我们交易团队最近也在AI效能提效，那我们与行业的佼佼者做提效的AmazonQ的产品 GenAI 相比，能否进行借鉴，结合自己的场景进行突破？那对于我们交易链路完成的需求迭代过程中，哪些是可以突破的点呢？**首先需要明确的是如果一个问题在舒适区里没遇到任何困难，说明价值不大，一个问题走出了舒适区很远才走通，回过头来看，可能是你迷路了进入了危险区。**所以在AI的时代，当迷茫时就应回到原点，或者退一步再思考。我们要找到解决问题的最短路径，才是整个刨根问底、追根溯源过程中最有价值的一环。

也许答案就在身边

我们认为新需求的「Vibe Coding」氛围编程，主要依赖于大模型的能力，我们在不依赖于业务场景的需求提示词工程完成过程中，可能今天我们做的优化，在明天就成为了大模型本身结合RAG进行反馈纠偏的能力，这种场景称之为业务场景的**浅水区**，属于**低价值领域**。同样，想在目前的上下文窗口限制下，完整实现在需求的规划、分解、执行全流程完成端到端上线发布到生产环境的场景，属于业务的**深水区**，复杂性高，属于**危险区域**。



作为一线的研发，最清楚研发日常工作中哪些工作的完成可以提升自己的研发效率，我们最直接的渴望就是AI可以直接帮我们生成可发布的代码 ，但是**什么样的场景是我们当前阶段应该去突破的呢？**资源总是有限的，我们要学会控制自己的欲望，我们要清楚时间最终花在哪儿，结果就会在哪儿，结合GenAI的提效区可以看到日常的非业务需求，如安全漏洞修复、星环预警处理、遗留AB实验下线、历史Switch开关治理，这些我们非常熟悉的场景，多少次在我们业务繁重时时不时的来上一个超时任务，不得不让我们投入琐碎时间分心去处理。这种工作对比起需求的端到端发布、架构级别的应用重构来说，实现难度和复杂度更低，但是，普及率和对于研发效能的提升更具普适性。



技术是帮助业务解决问题的手段，业务的形态，技术的使用是多样的，但是沉淀解决问题的能力，抽象解决问题的方法是不会变的。在当前阶段，在业务维度：每一个动作流的落地都可以解决当前研发阶段的一个痛点问题，在通用维度，每一个中间件MCP能力的接入都能直接的影响到最后模型对于业务上下文的理解，**RAG的准确性又关系到该工作流的落地的可行性，**一个

准确性低于50%的AI生码模型，我们认为是灾难性的，因为我们在后续的CR和质量保证环境投入的精力将会比提效的能力还要多。

我们认为只有**业务和通用能力的双重建设**，选取足够细分的场景，只要一件小事情的能被90%正确性以上的解决，那这个方案我们认为是可被接受、安全且具有推广价值的AI提效途径。

02 痛点分析

在S1开始阶段，我们深入调研交易链路一线研发的日常工作场景，系统性梳理出多层次的效率痛点。交易链路的由于复购、场景购、逆向、服务、物流每年各种新特性需要在AB实验上新增实验，实验的完整分流逻辑需要结合代码和平台双重验证才能知晓线上的分流逻辑，与此相似的还有数以万计Switch开关，**一旦出现线上问题，全链路上的实验和Switch开关在预发和线上的不一致，验证制约了线上问题的复现和定位**。同时，要清理这些逻辑上垃圾，时常需要将研发中抽离出来，这些机械性且缺乏业务价值产出的需求严重制约了研发效率，日志格式规范、代码注释标准、错误码管理等领域。

针对上述核心痛点，我们采用递进式策略，优先从日常高频重复性工作入手，重点聚焦AB实验推全下线和Switch开关治理两个典型场景。**通过AI自动化生码技术实现变更生成与发布，提升研发治理效率。**

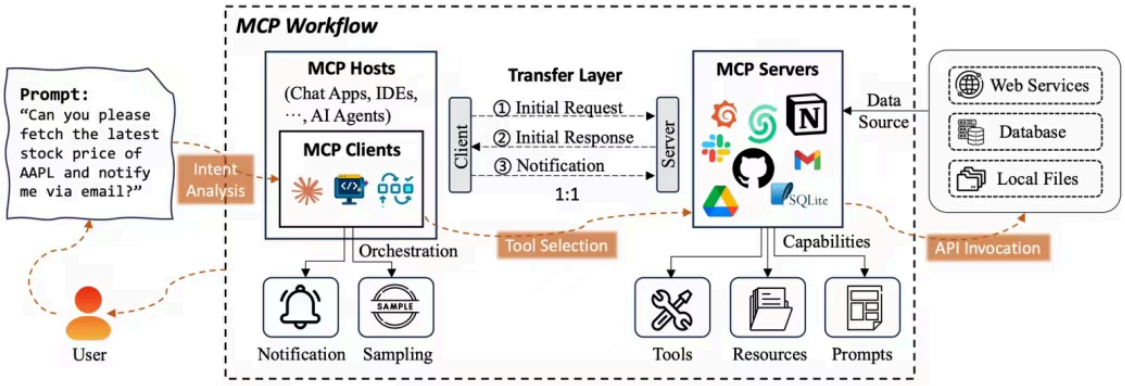
- AB实验推全或下线 – 实验结束后需要精确处理分流逻辑，避免代码架构污染
  - 获取实验全生命周期信息，结合研发实际诉求和代码库上下文进行智能生码
- Switch冗余开关治理 – 线上环境与预发环境配置不一致导致问题排查困难
  - 整合线上预发开关状态，基于MCP平台切流策略建议，实现开关物理切流

问题描述	依赖能力	优先级
AB实验影响代码架构	**CodeBase，AB实验**	**交易独有场景，重要且紧急**
Switch冗余开关治理	**CodeBase，Switch**	**交易独有场景，重要且紧急**
星环日志告警治理	CodeBase，星环日志	通用场景，重要不紧急
安全漏洞修复	CodeBase，知识库	通用场景，重要不紧急
错误码管理混乱	CodeBase，知识库	交易场景，重要不紧急

都说万事开头难，**AI的时代需要改变思维，立刻开始做**，才是生产力变现的关键。确立了以**「 通用MCP能力基座为基础，集合业务问题场景所需的工作流步骤和上下文，实现精确到点需求发布，是我这次选择的目标 」**走一步看一步也不是贬义词，小步快跑，敏捷迭代，一边执行，一边思考，遇到问题解决问题的过程，在AI大模型的实践红海中，需要以最快速度拆解出目标完结的最小闭环。

工作流的设计

- MCP workflow



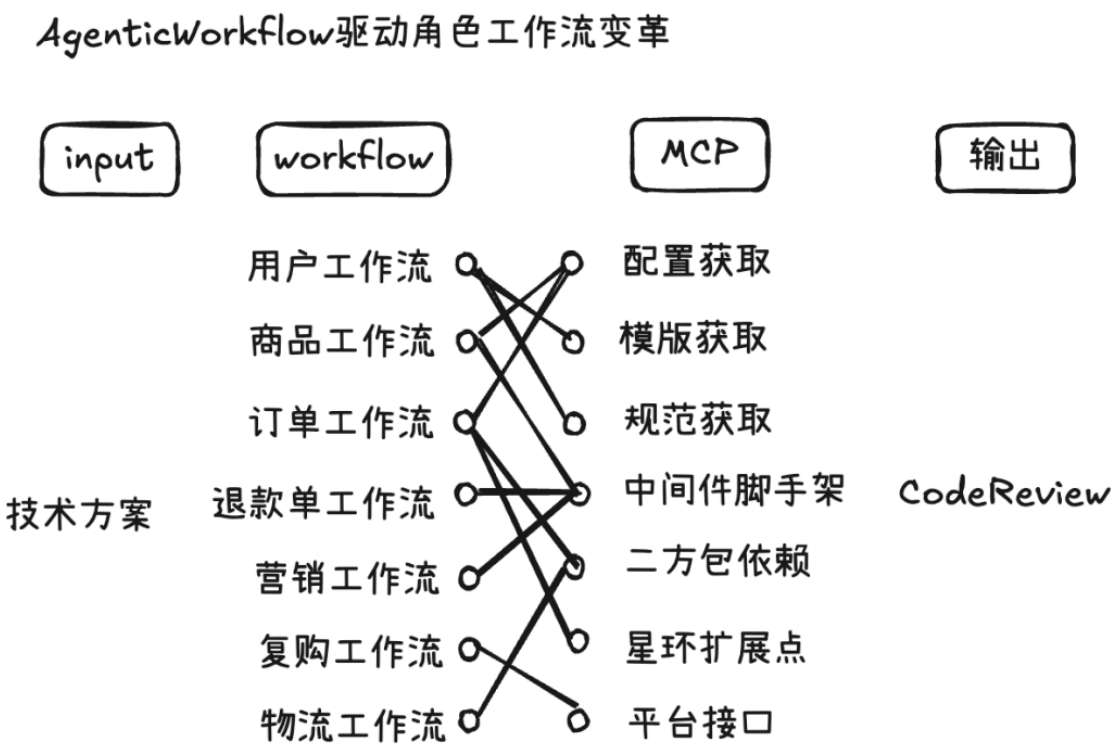
以通用场景的查股票来进行举例：

- 智能体通过Client获取各Server注册的工具清单。
- 用户提出"查股价+邮件通知"复合需求触发任务。
- 大模型解析语义后联动选择股票接口和邮件工具。
- Client将工具调用指令路由至对应功能Server。
- Server异步执行API查询及邮件推送操作。
- 执行结果经Client回调至智能体完成服务闭环。

我们工作流设计之初，就是参考现在比较成熟的MCP Workflow 执行流程，主要考虑从通用场景和专用领域场景的转换和映射。

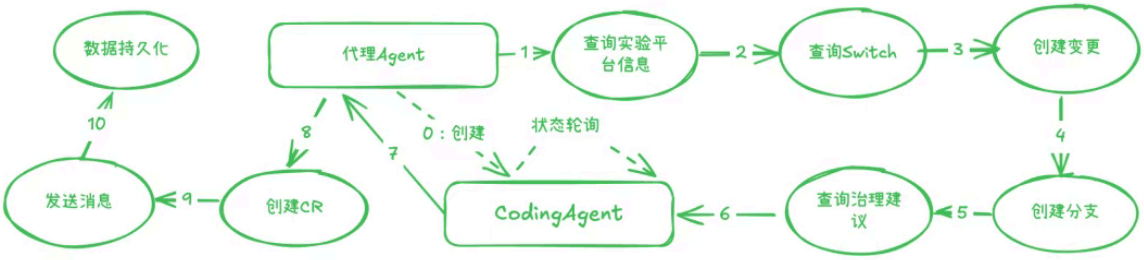
- Agentic workflow

Agentic workflow 是对 MCP workflow 的智能化升级，如同**智能印钞系统**不仅具备基础打印能力，更通过多Agent协同实现全链条闭环：当基础打印机（MCP workflow）按预设模版输出纸币时，Agentic体系会自主触发**防伪检测、自动分装、流通追踪**等子工作流，实时协调质检规则引擎、物流调度模块与区块链审计系统，在保障效率的同时动态优化全局风险控制与资源分配，最终实现复杂场景下“**执行–验证–优化**”的自驱式闭环。



体验过不同 Agent 流程编排会发现，Workflow 最终会成为一个组件被调用，同时workflow 中又能够嵌套新的workflow，实际上不管是基础节点、插件工具、LLM、逻辑条件处理等，**都实际上是一个以输入、输出的组装的模块，不同的组件之间通过连接构成一个更大的模块**。基于独立的输入输出的模块，结合起来，在每个域都可以独立成完整的工作，这样可以构建更复杂的需求实现。

- AB workflow



AB实验下线完整工作流

研发场景中的 AB实验下线、Switch开关治理、星环告警修复都属于抽象成相对比较独立的工作流，以我们的AB实验下线的完整流程举例，在我们在传统的电商业务业务场景中，相关领域需求研发的工作流都可以最终**拆分到一个一个MCP执行获取的模版代码，再结合codebase和业务上下文，最终封装成完整的提示词，完成整体的领域工作流的研发实现。**

模型能力调研

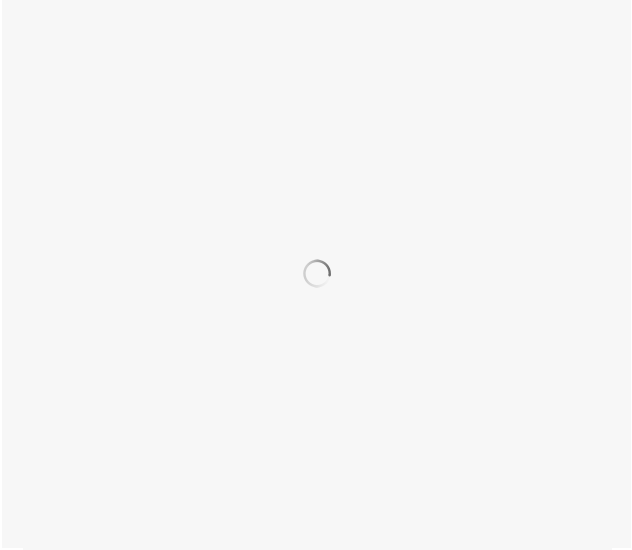
- Claude4模型

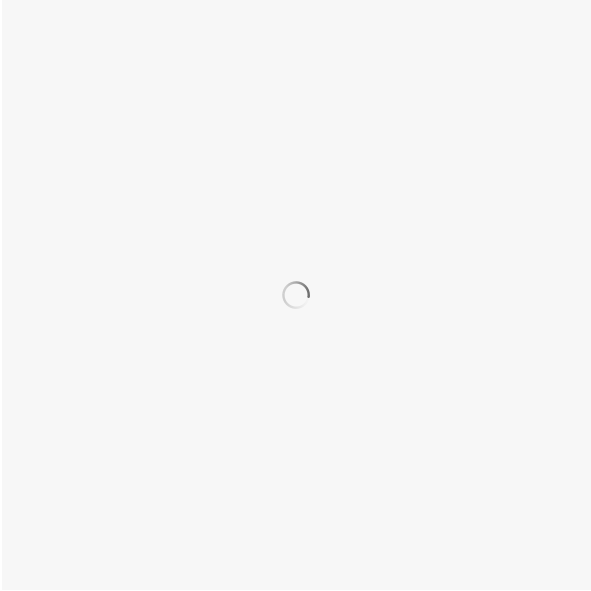
目前编程圈中最火爆的应当是由Anthropic开发的Claude 4模型，该模型具备了混合推理能力和强上下文能力，兼具即时响应与深度思考两种模式，并可以根据任务需求在两种模式间切换：

- Claude Sonnet 4 系列模型具备强大的工具使用能力，兼顾效率与能力提升
- Claude Opus 4 特别在记忆能力上有重大突破，对长时间任务的场景中有稳定的发挥

对比维度	Claude Opus 4	Claude Sonnet 4
持续工作时长	可连续工作数小时（如Rakuten验证的7小时独立任务）	平衡性能与效率，适合内部和外部用例
核心能力	代码质量提升、复杂代码库理解、高级AI代理开发	日常编码辅助、多功能应用开发、代码审查
最佳应用场景	大型代码库重构/优化、深度研究、长时间复杂任务	高效开发、问题修复、内部/外部用例支持

基于Claude4 模型的全新终端AI编程工具ClaudeCode，可以通过自然语言指令帮助开发者高效率地完成代码编写、调试和项目管理任务。它直接集成在开发者的工作环境（如终端）中，无需依赖额外服务器或复杂配置即可运行。





实际使用下来，ClaudeCode能够在复杂推理任务中表现出色，同时保持交互体验的流畅性。随着Claude Code的爆火出圈，这个**产品是真实地提高程序员的开发效率**。但是随着Cursor、Claude Code对国内ip加大力度管控，使用这类前沿工具变得十分困难，国内的程序员不得不投向更多的国产模型。

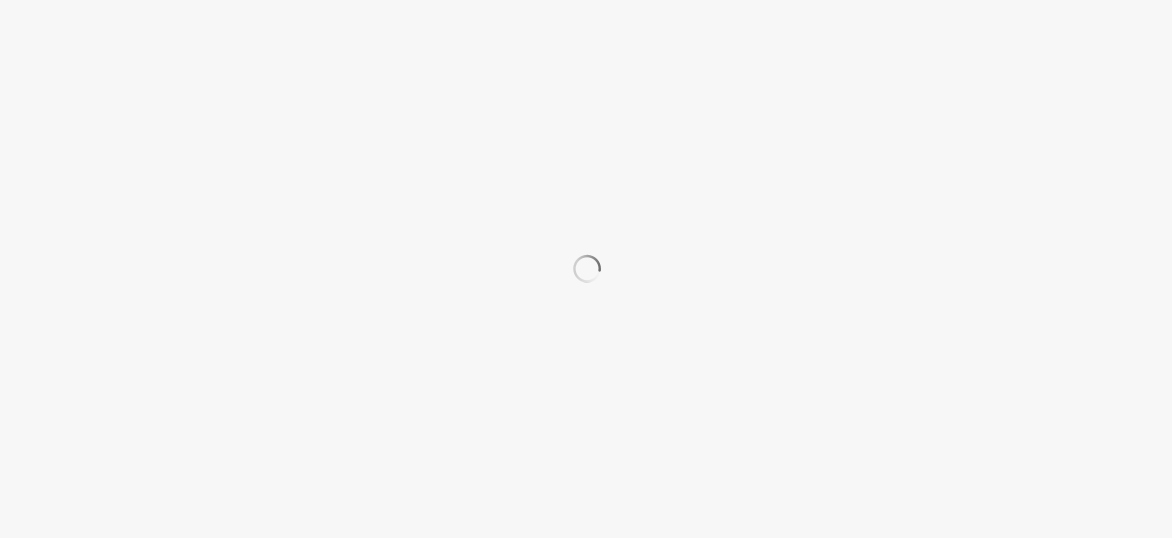
- QwenCoder模型

最近一个月也迎来了很多惊喜：**Kimi K2**、**Qwen3-Coder**、**GLM-4.5** 等国产大模型相继涌现。国产编程工具正不断突破创新，致力于为国内开发者打造更智能、更高效的极致开发体验。Moonshot AI刚在11号发布了旗舰模型Kimi-K2-Instruct；通义千问就在22号发布了Qwen3-235B-A22B的**非思考模式版本——Qwen3-235B-A22B-Instruct-2507**；紧接着，23号又立马推出了通义迄今为止**最具代码能力的代码模型Qwen3-Coder**，甚至还在下班后搞“突袭”，发布了**思维和推理能力更强的Qwen3-235B-A22B-Thinking-2507**。

模型名称	总参数量	上下文长度	SWE-bench Verified	LiveBench (Average)	备注

Kimi-K2-Instruct	**1T**	128K	65.4%	Coding 71.78% Agentic Coding 20%	数学推理强 NMLU 89.5%
Qwen3-Coder	480B	**1M**	69.6%	Coding 73.19% Agentic Coding 25%	原生256K上下文可拓展至1M

SWE-bench Verified检查了模型们**解决实际代码问题**的能力，看起来**Qwen3-Coder略胜一筹**，整体能力可与Claude Sonnet 4 （得分70.4%）媲美。如此分析，在BUG修复或功能实现类任务中，Qwen3-Coder效果会更好一些，尤其是在代理编码任务上。不过虽然**Qwen3-Coder在“纸面成绩”上更强，但实际开发中遇到的困难可能与测试完全不同。**



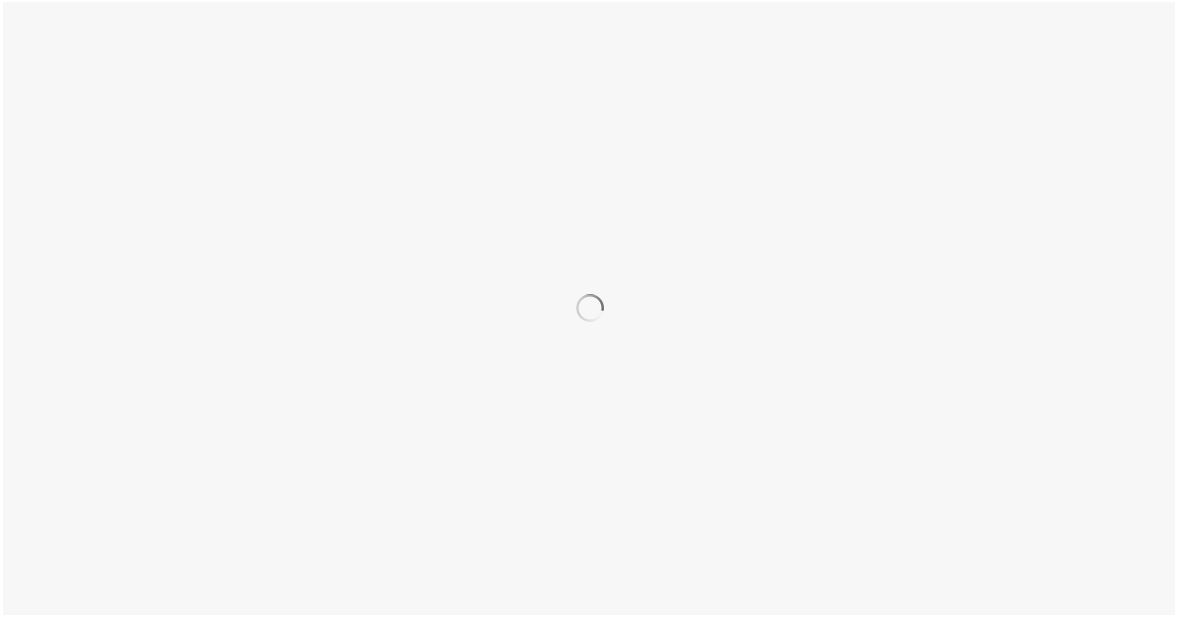
总体来看，Qwen3-Coder 在短期任务中表现出色，对项目需求的理解较为完整，适合生成逻辑复杂、效率要求高的代码。但在大型生产环境中，其稳定性与扩展能力略显不足。相比之下，Kimi-K2 表现更为稳健，擅长将复杂任务拆解为子任务逐步执行，生成的代码可读性和可用性都很高。每种模型都有其优势与局限，关键在于根据实际场景选择合适的工具。如果**追求可控性、高效率和输出质量，Kimi-K2 是更优选择；而若侧重复杂逻辑建模与高效编码，Qwen3-Coder 则更具优势。**

从准确率的角度来说，Claude4无疑是最佳的选择，但鉴于对于公司C3级别代码仓库的安全考虑，我们应该使用Qwen3-Coder作为高安全级别的仓库的兜底选择。

CodeAgent选择

Agent就是AI作用于互联网的软件形态产品。**如果说PC时代是网站，手机时代是APP，那AI时代就是Agent**。所有的领域，所有的应用，可能都会被AI通过Agent革新一遍。下图展示开源

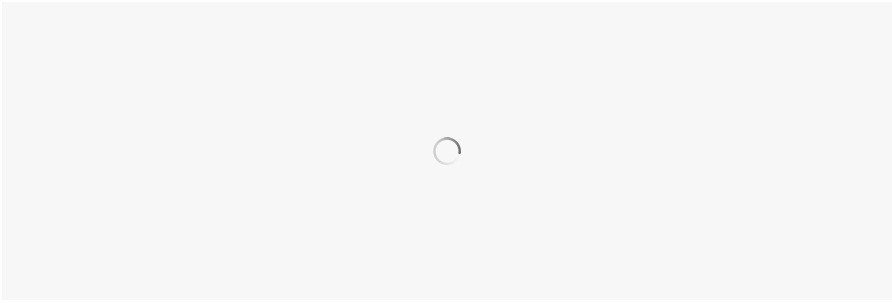
的CodingAgent应用可以说是百花齐放：



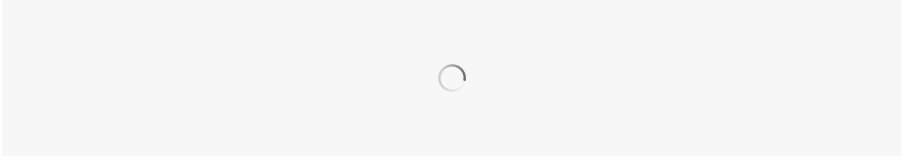
Agent 正在迅速普及，似乎正在取代我们熟悉的Chatbot，那么我们平时工作已经或多或少的使用的CodeAgent究竟是什么？

- 什么是LLMAgent

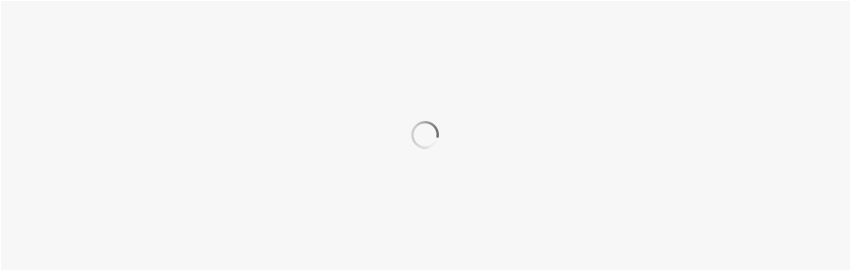
它其实本质上是增强型LLM Agent的一种实现。要理解 LLM Agent 是什么，让我们先回顾一下 LLM 的基本能力。传统上，LLM 最基本的功能只是预测下一个 token。



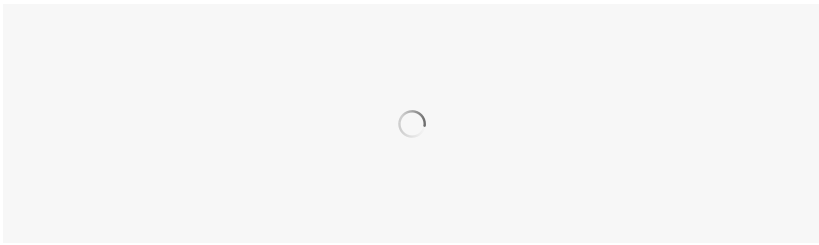
最基本的就是通过连续采样多个 token，我们可以模拟对话，并让 LLM 对我们的查询给出更长的回答。



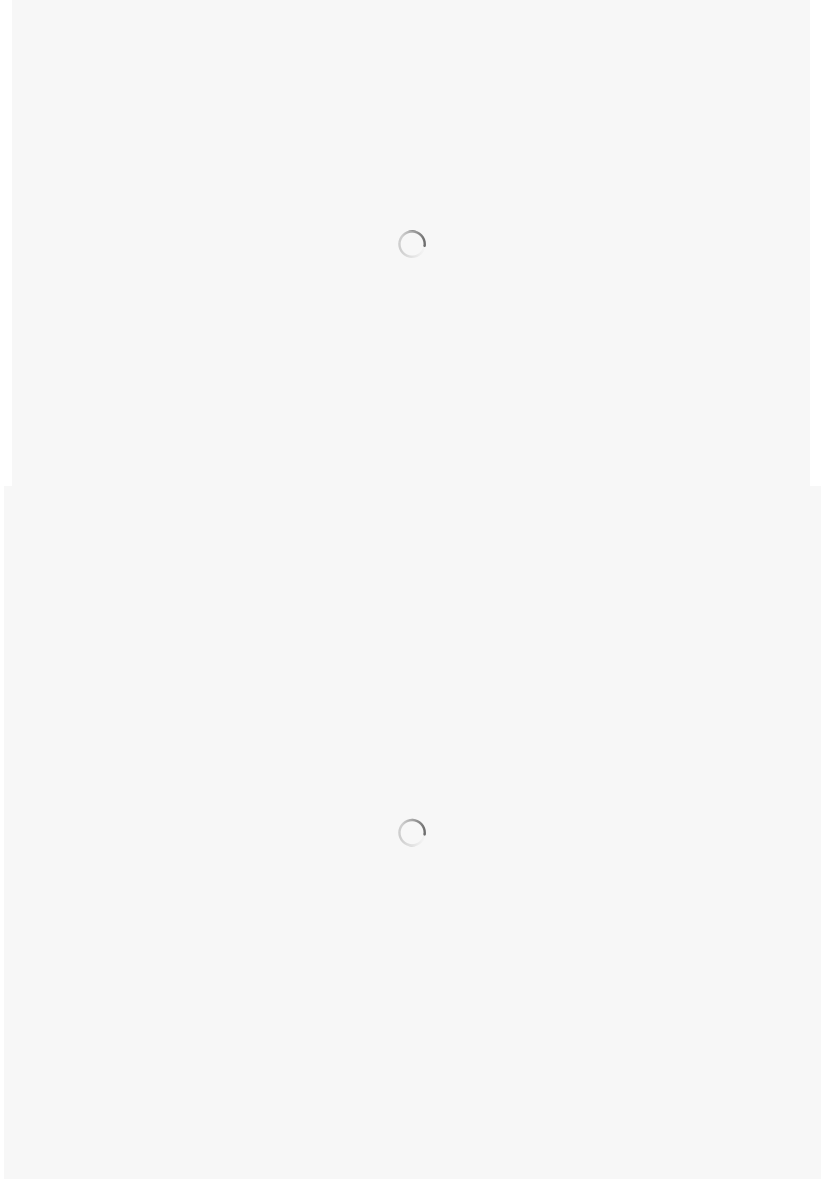
然而，仅仅有LLM的能力是否就能称为Agent呢？并不能！比如当我们继续进行“对话”时，它不记得之前的对话！



这也是为什么LLM的Agent都会有基于记忆产生知识库的产物：如Rules、Docs、记忆、Wiki，加上记忆以后，我们可以称之为Copilot了，但是Agent的定义是需要根据用户的指令产出完整的方案或者代码。比如，LLM在很多其他任务上如乘除法、git命令、代码仓库检索等方面表现不佳：



这时候就需要通过外部git工具、wiki记忆和code检索系统来弥补它们的不足。通过外部系统，LLM 的能力可以得到增强。Anthropic 公司将这称为“增强型 LLM”（Augmented LLM）。例如：我们日常使用到的CodeAgent，就是封装了很多基于codebase能力引入的工具。

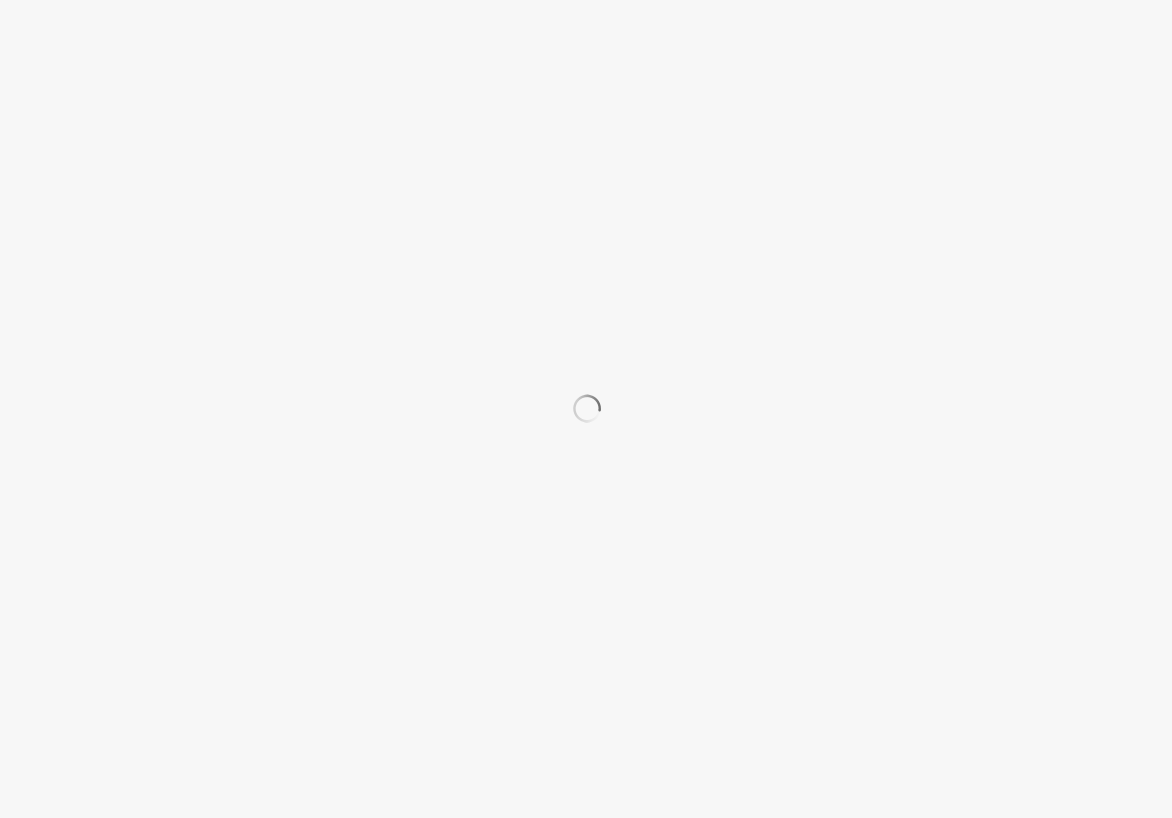


这些工具能力不仅CodeAgent之间可以互相调用，也可以封装成mcp服务，直接被外部系统调用。

通过规划、记忆、工具的补充是不是增强型的LLM是不是我们上面看到CodeAgent，还不完全是！

根据 Russell & Norvig, AI: A Modern Approach (2016) 文献显示：

Agent 的准确定义可以被看作是能够通过传感器感知环境，并通过执行器对环境采取行动的任何事物。



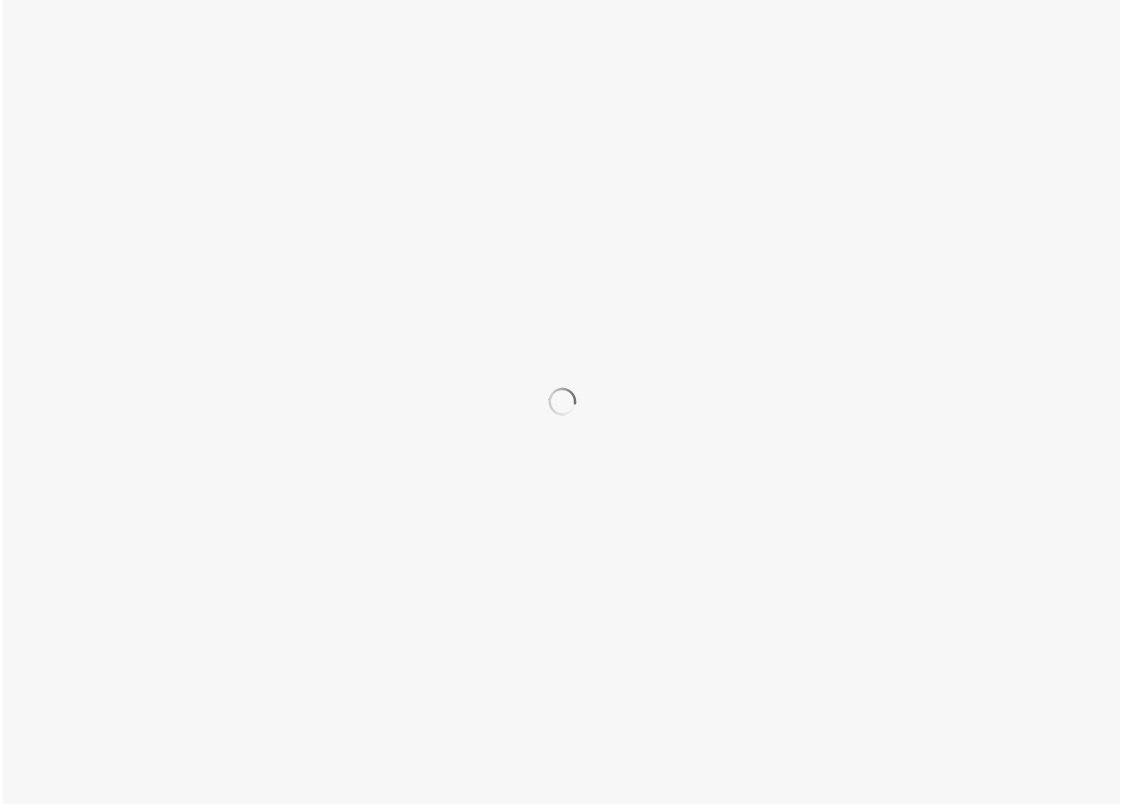
完整的Agent 与需要与环境进行互动，以CodeAgent举例，通常包含几个重要组件：

- **环境 (Environments)** — 仓库上下文，@Codebase
- **传感器 (Sensors)** — 游标cursor，定位所需修改代码的位置
- **执行器 (Actuators)** — @grep_search @search_replace 工具
- **效应器 (Effectors)** — 大模型本身所具备的 Planning & Relection

具备LLM的决策规划能力，并能够通过传感器感知环境，并记录历史上下文信息，并通过执行器使用工具对环境采取行动的代理，可称为完整的agent。

- Agent的决策逻辑

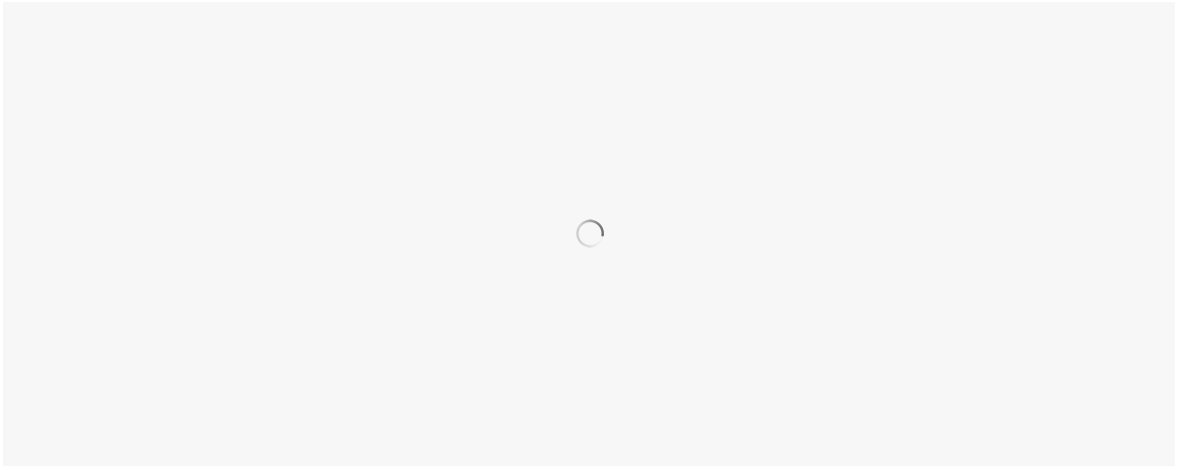
Agent的核心决策逻辑是让LLM根据动态变化的环境信息选择执行具体的行动或者对结果作出判断，并影响环境，通过多轮迭代重复执行上述步骤，直到完成目标。



精简的决策流程：P（感知）→ P（规划）→ A（行动）以我们所依赖的CodeAgent举例：

1. **感知（Perception）**从上下文、wiki、Rules、记忆获取提取相关知识的能力。
2. **规划（Planning）**是指Agent为了某一目标而作出的决策过程，复杂场景会以待办任务的进行罗列。
3. **行动（Action）**是指基于环境和规划做出的最终的Action：如修改代码、编译、单元测试等。

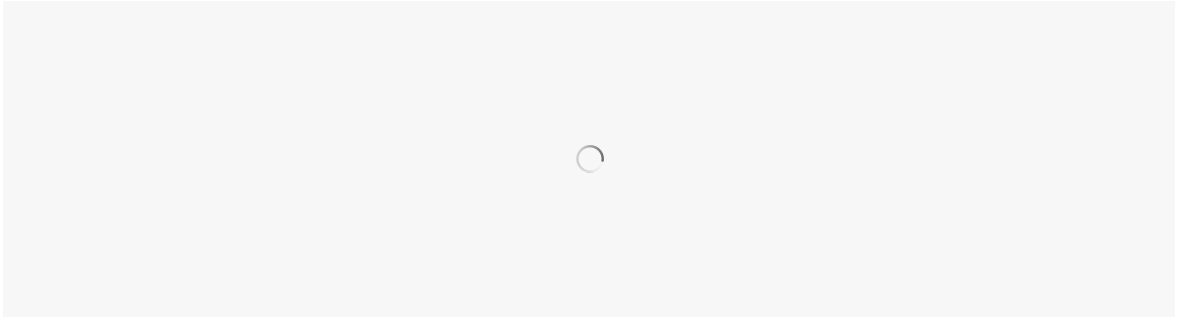
其中，Policy是Agent做出Action的核心决策，而行动又通过观察（Observation）成为进一步Perception的前提和基础，形成自主地闭环学习过程。工程实现上可以拆分出四大块核心模块：**推理、记忆、工具、行动。**



影响模型最终结果的是决策模型的框架，目前Agent主流的决策模型是ReAct框架，也有一些ReAct的变种框架，以下是两种框架的对比。

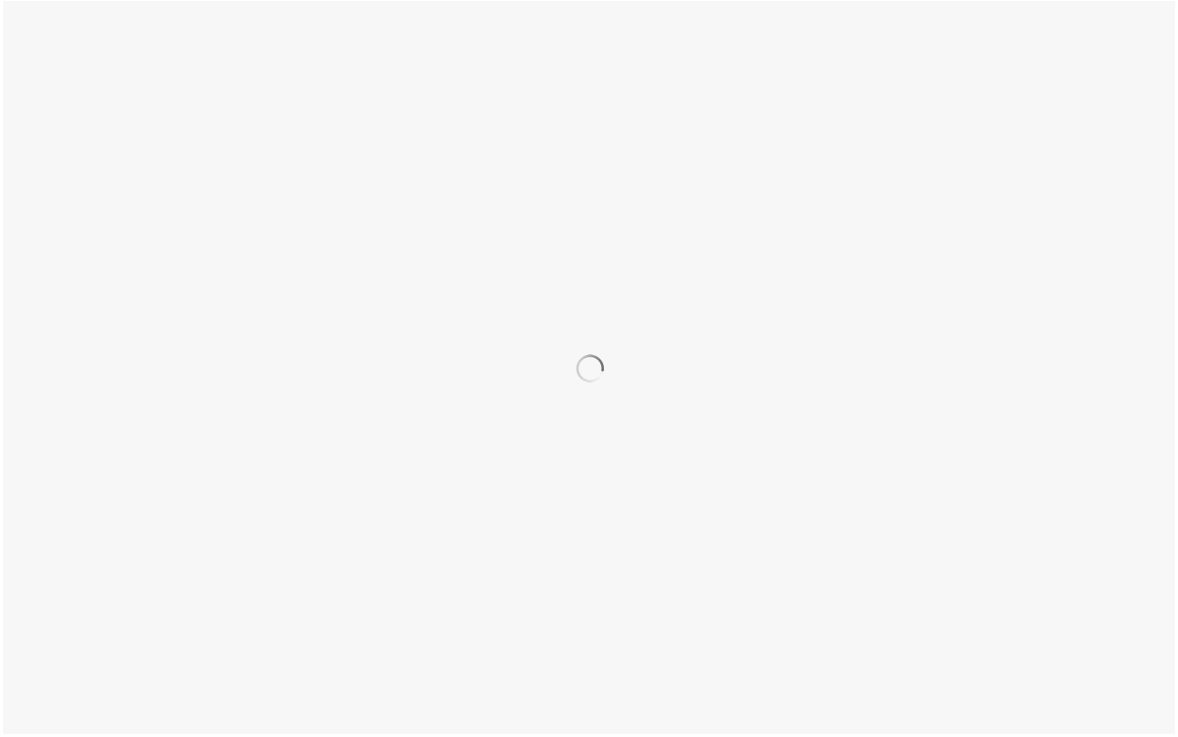
- **传统 Reason and Act**

ReAct=少样本prompt + Thought + Action + Observation 。是调用工具、推理和规划时常用的prompt结构，先推理再执行，根据环境来执行具体的action，并给出思考过程Thought。



- **Plan-and-Execute ReAct**

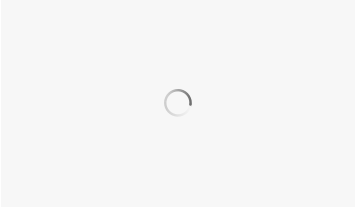
一部分Agent通过优化规划和任务执行的流程来完成复杂任务的拆解，将复杂的任务拆解成多个子任务，再依次/批量执行。优点是对于解决复杂任务、需要调用多个工具时，也只需要调用三次大模型，而不是每次工具调用都要调大模型。缺点也显而易见，在简单任务中需要额外的进行规划和拆解需要更多的耗时。



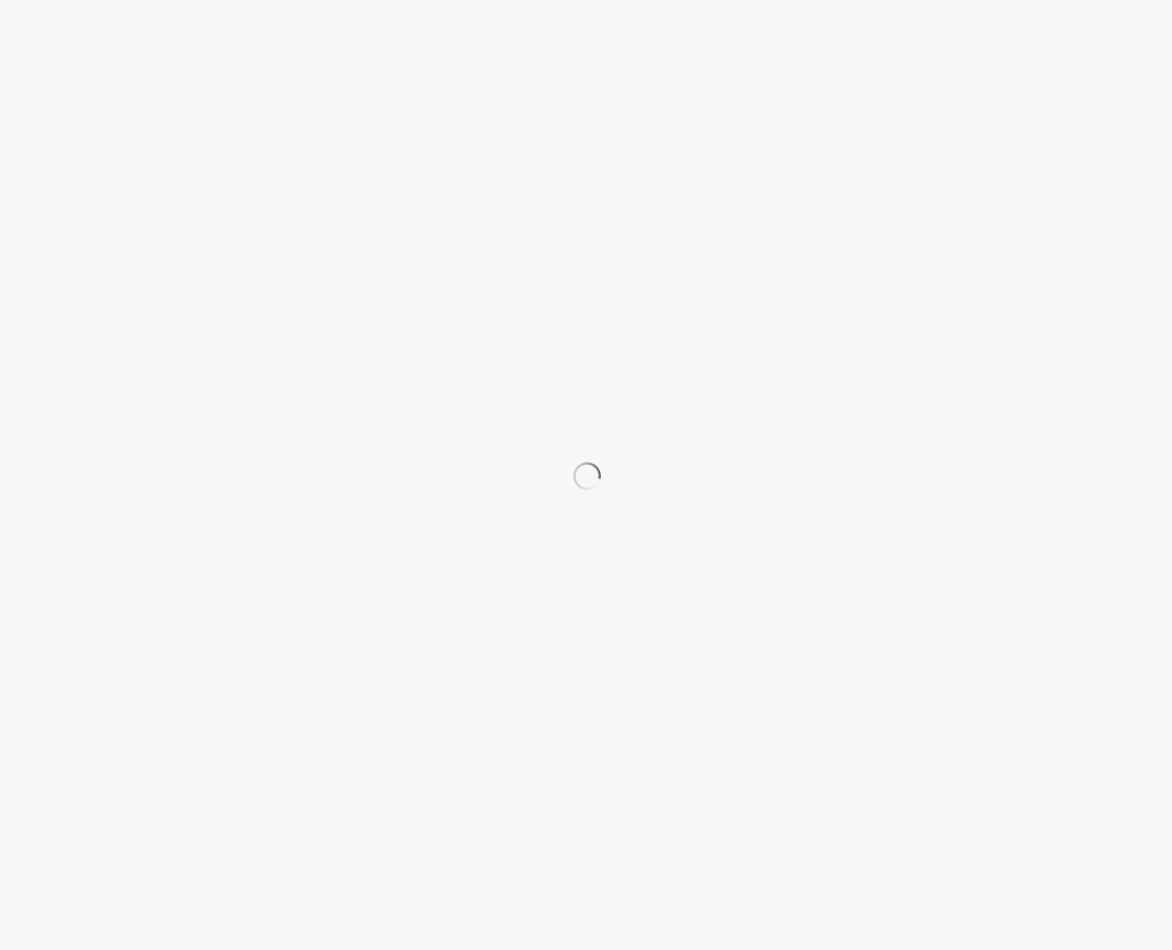
基于以上的分析，我们的目前阶段工作流模型只需基于ReAct模型即可完成我们预期的诉求。

- Agent常见框架

在Agent框架选择上，一般我们在IDE或者cli中直接使用的都是单一Agent，单一LLMAgent可以通过工具、记忆、规划执行，很好的完成我们诸如：代码变更的诉求，如果我们的工作流有且仅有代码修改时，这样的架构无疑是最高效的，但是，我们也必须看到单一 Agent 有几个问题：工具太多可能导致选择复杂，上下文变得过于庞大，以及任务可能需要专业化。

	**多智能体优点：** 1. 复杂问题拆解：每个子agent负责解决特定领域的问题，降低对记忆和prompt长度的要求； 2. 开闭原则：通过增加子agent来扩展功能，解决单agent并发的问题 3. 可操控性强：可以自主的选择需要的视角和人设更快的解决问题 **缺点：** 1. 成本和耗时的增加； 2. 交互更复杂、定制开发成本高； 3. 简单的问题单Agent也能解决；

如交易场景：不同业务领域的Agent所需要的上下文是完全隔离的，同时不同场景Agent在完整需求交付链路上所扮演的角色也完全不同，因此，我们对于未来的设计规划可以转向**多Agent**，这是一种框架，其中多个 Agent（每个 Agent 都具有**工具**、**记忆**和**规划能力**）相互之间以及与环境互动：



结合我们现有工作流的场景，CodingAgent除了代码修改以外，虽然也可以做如变更创建、分支创建、CR创建等等MCP工具的使用，但是实际体验来说，使用一个前置代理Agent可以更快捷、高效、低消耗的完成同样的事情。未来，当知识库、规范足够大时，Multi-Agent更能凸显出对于特定领域记忆和上下文窗口的适配性。

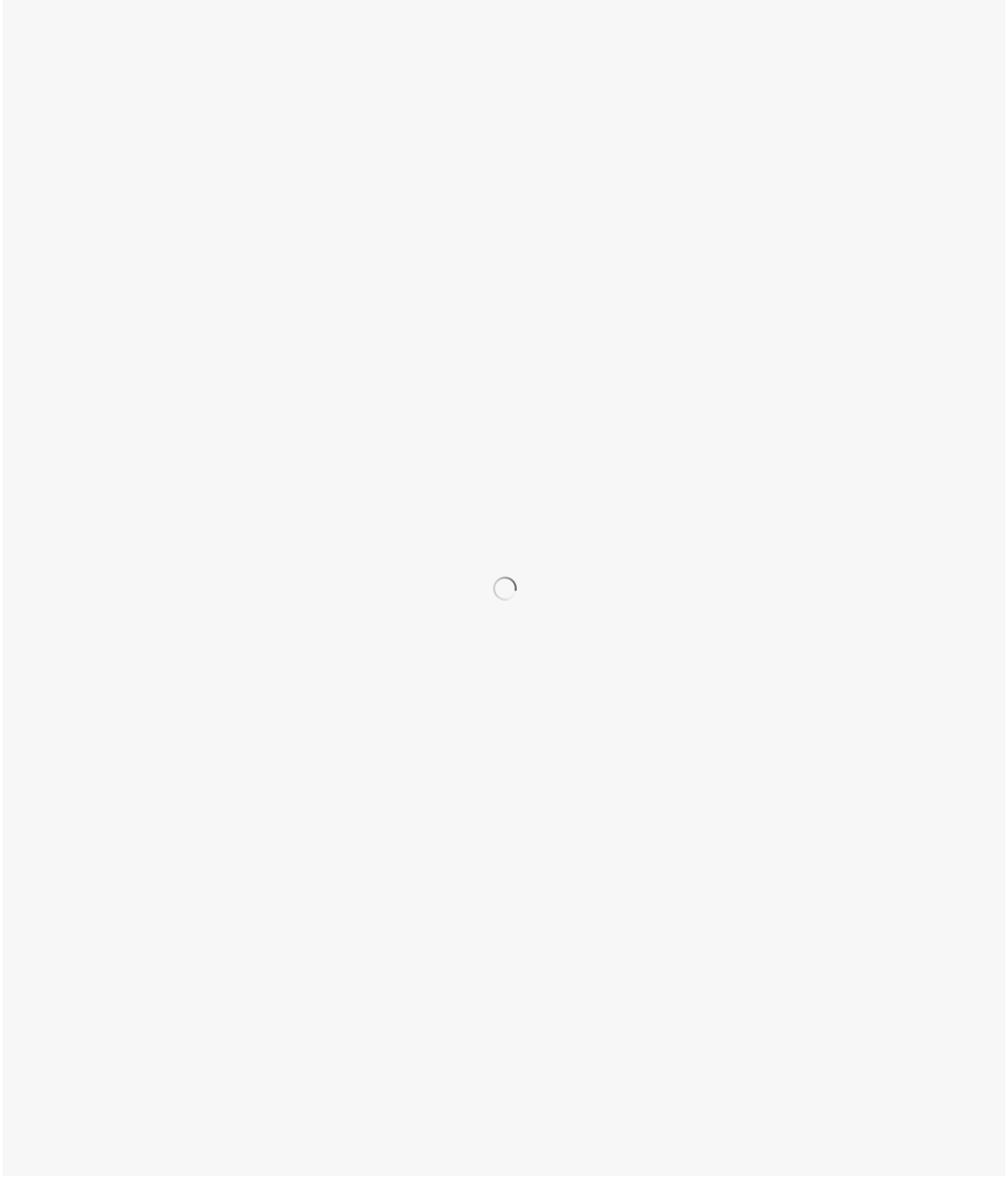
但Multi-Agent更多还是为了解决当前LLM的能力上缺陷，通过LLM多次迭代、弥补一些显而易见的错误的过渡产物，不同框架间仍然存在着极高的学习和开发成本。始终相信一个技术的价值，不在于你的方案有多先进，框架有多厉害，而是是否可以更好的满足业务的需求，创造出业务独有的范式，即使我们把现在的问题放大十倍，也会发现很多事都不是现在该关心的事。在当前阶段我们选这个Single-Agent框架 + 前置代理Agent就可以完成全部的诉求。

协议选择

技术的目的是帮助业务解决问题，我们要非常清楚我们问题的定性，选择最合适的技术。

- MCP协议 – 智能体与工具通信

Anthropic是一家成立于2021年的人工智能初创公司，由OpenAI的前研究副总裁Dario Amodei和GPT-3论文的一作Tom Brown等人共同创立。该公司非常有名的产品是大模型-Claude。24 年11月，Anthropic推出MCP 协议，并在自家的 Claude 客户端做了支持。后续各大模型公司纷纷开始支持MCP协议。



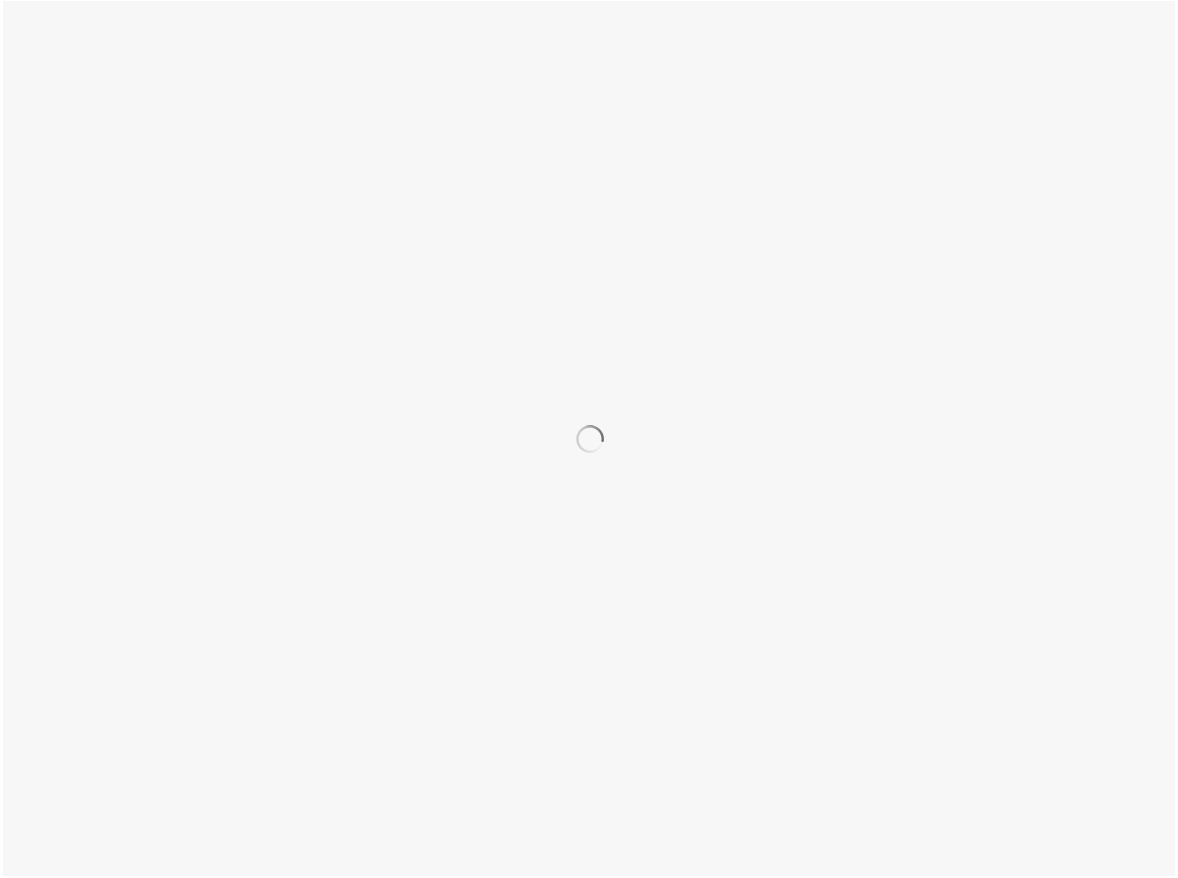
从上面的动图我们可以看出，完整的MCP架构由三部分组成：

- Host 是集成AI工具数据的应用平台（如豆包APP），托管Client并提供AI任务执行环境；
- Client作为通信中介，管理Host与多Server间的请求响应、通知处理和性能采样；
- Server提供三大核心能力。
    - 工具模块调用外部API简化服务交互；
    - 资源模块打通数据库实现数据驱动决策；

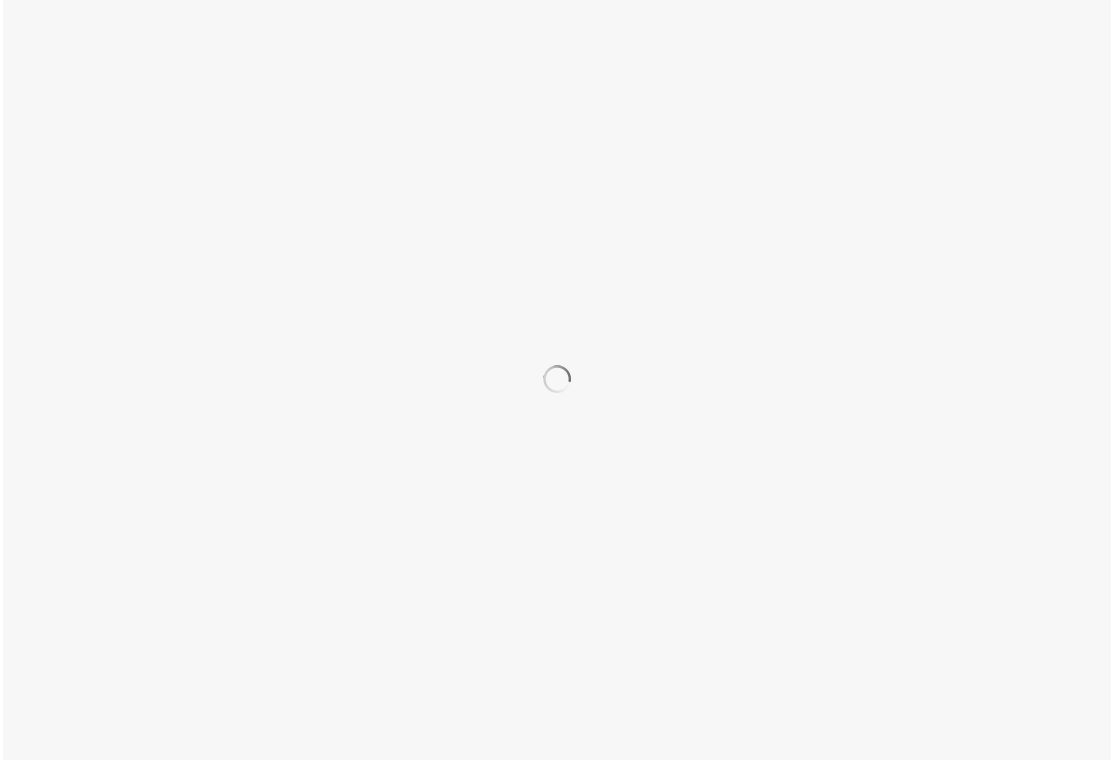
- 提示词模块通过模板化指令优化AI工作流程。

- A2A协议 – 异步执行

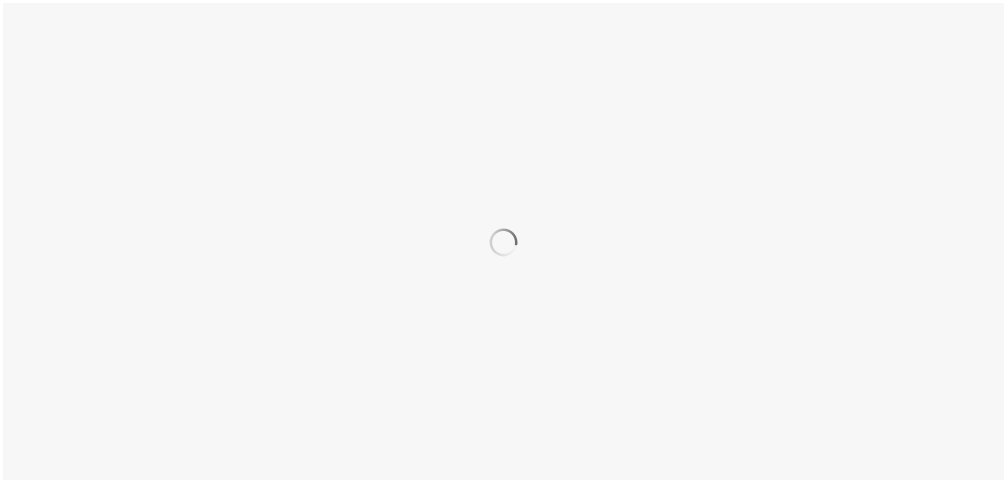
我们编码需求实现的过程中，需要和 DevStudioAgent、AoneAgent等专职负责代码修改的CodingAgent进行通信，同时，在特定场景中又需要和相关CodingAgent底层提供的SandBox的MCP编码工具进行直接交互，最后实现我们的完整诉求。



那我们Agent之间应该如何通信呢？答案就是A2A协议，Agent2Agent（A2A）是 Google 推出的一个开放通信协议，专为多个 AI Agent 之间的协作而设计。它允许来自不同平台、不同厂商的智能体像人类一样进行对话、协商、分工与协作，从而共同完成复杂任务。你可以将它类比为网页世界的 HTTP 协议——**A2A 是 AI 代理之间的“通用语言”**。



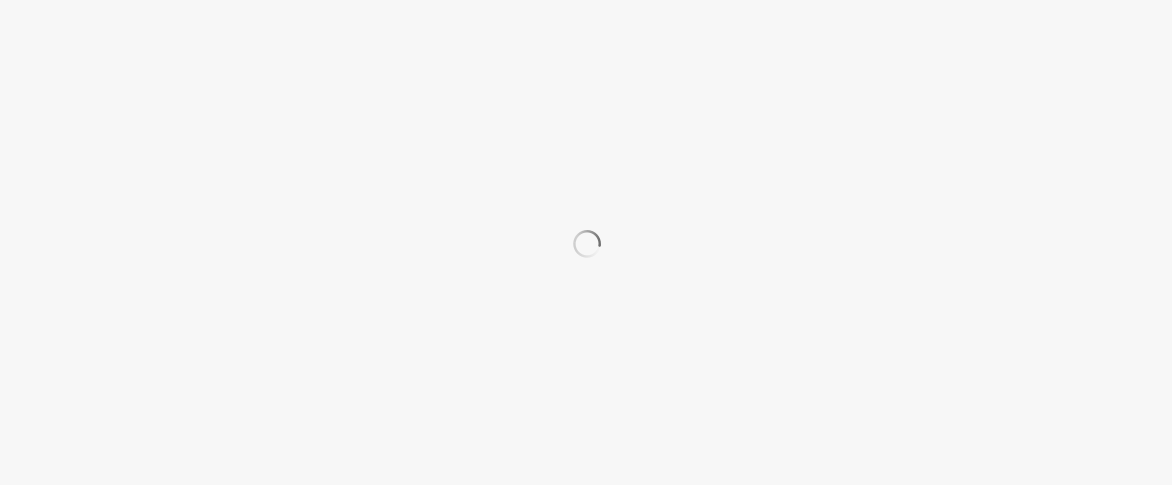
A2A 的核心目标是构建一个“智能协同体”（Intelligent Collective），让多个 Agent 像流水线一样协同工作，提升任务执行效率与智能系统的整体表现。虽然 A2A 与此前 Anthropic 提出的 MCP有相似之处，但两者的定位和功能存在明显差异，简而言之，MCP 更像是“Agent 与工具”的接口，而 A2A 是“Agent 与 Agent”的协作协议。



- AG-UI协议 – 实时展示

考虑到服务端在执行工作流的任务时，是一个相对比较长的过程。如何让用户在工作流处理的过程中实时感知到整体的进度？如何让模型规划（Planing）的过程，模型使用工具执行操作的过程变得可视化？用户与这些智能体之间的互动是否有统一的标准？答案：有！那就是AG-UI（**Agent-User Interaction Protocol**）协议，AG-UI 是由 CopilotKit 发布的开放、轻

量、基于事件的协议，通过标准 HTTP 或可选的二进制通道，以流式方式传输 JSON 事件。它主要用于标准化 AI 智能体与前端应用程序的交互，确保实时同步和高效通信。

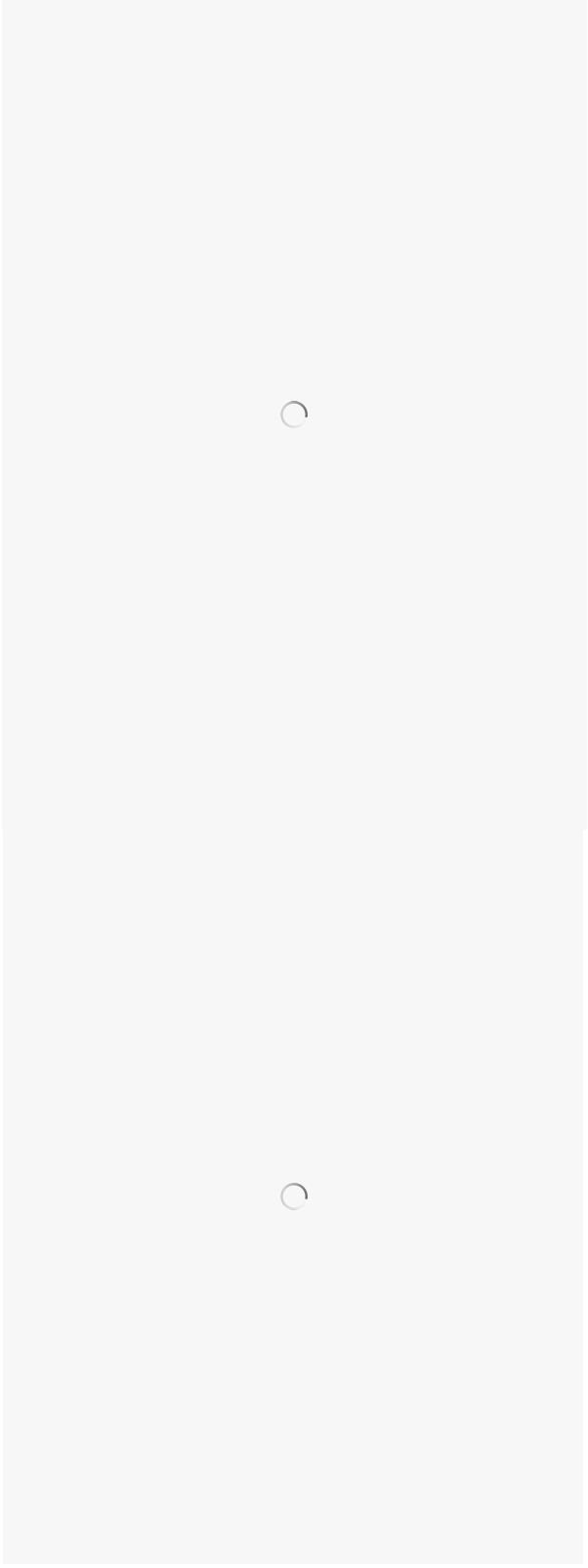


AG-UI 的出现，为用户与智能体之间的互动提供了标准，优化了用户体验。AG-UI 定义了一套小而强的事件类型集合，以 JSON 对象形式进行传输。以下为核心类型及结构说明：

类型	用途	示例 JSON 结构
lifecycle	通知任务开始、结束	{ "type": "lifecycle", "status": "started" }
text-delta	流式输出内容片段	{ "type": "text-delta", "value": "workflow" }
tool-call	Agent 意图调用工具	{ "type": "tool-call", "tool": "grep_search", "input": "AB_ExperimentKey" }
tool-result	工具调用返回结果	{ "type": "tool-result", "value": "done" }
state-update	UI 状态同步（快照/差异）	{ "type": "state-update", "snapshot": { "mode": "edit" } }

这个特性，使得AG-UI协议特别适合于多轮协同智能体系统（Multi-Agent Copilot）的页面表达的开发：

- AG-UI 与 Agent runtime（如 LangGraph）协同，管理上下文和 UI 状态
- 支持长任务生命周期、思考中状态提示、功能建议列表



完整生态

AG-UI 与 MCP（Model Context Protocol）和 A2A（Agent-to-Agent）协议共同构建了完整的 AI 代理生态系统：

- **MCP**：解决智能体与工具之间的通信。
- **A2A**：规范智能体之间的协作。
- **AG-UI**：专注于用户与智能体之间的互动。

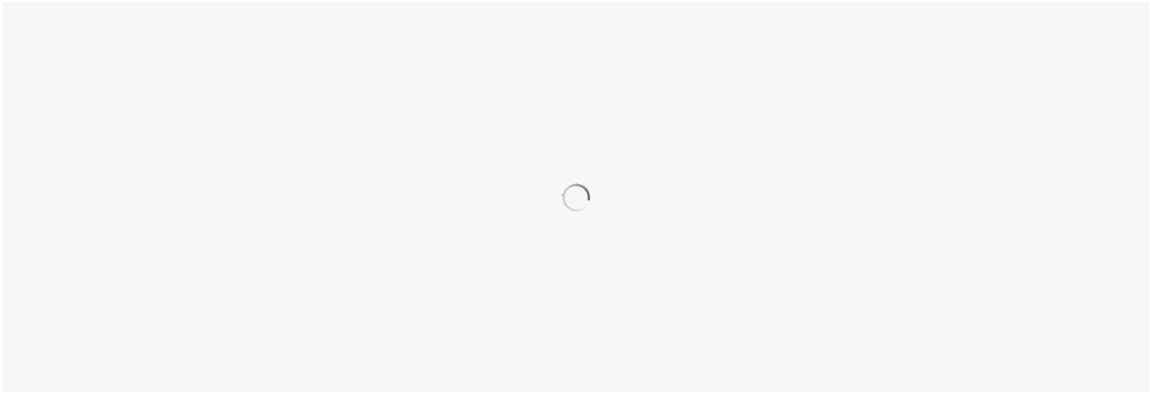


目标确定

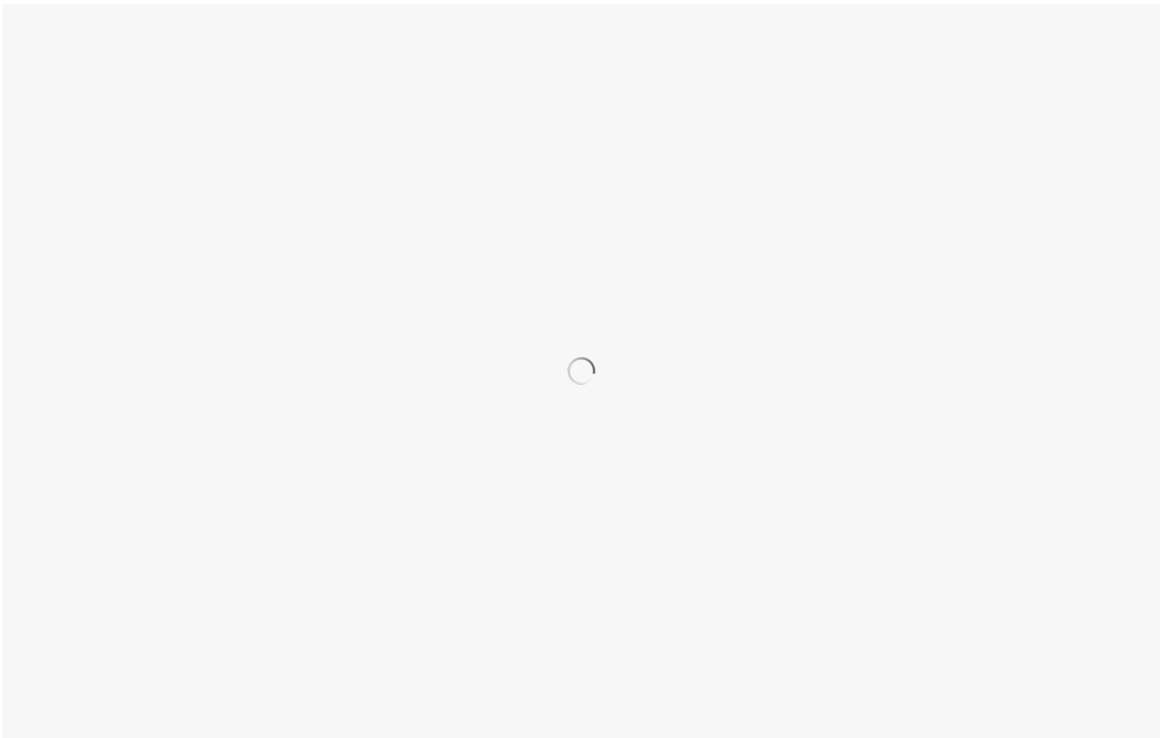
从AB实验下线推全这个轮子造起

- 什么是一休AB实验

没使用过相关平台可能还是不太理解。简单来说，就是通过分流以后的分组对比确定功能的效果。



对于研发人员来说，就是调用一休的sdk，一休的结果告诉我用户在哪个组，然后业务逻辑走不同的分支：

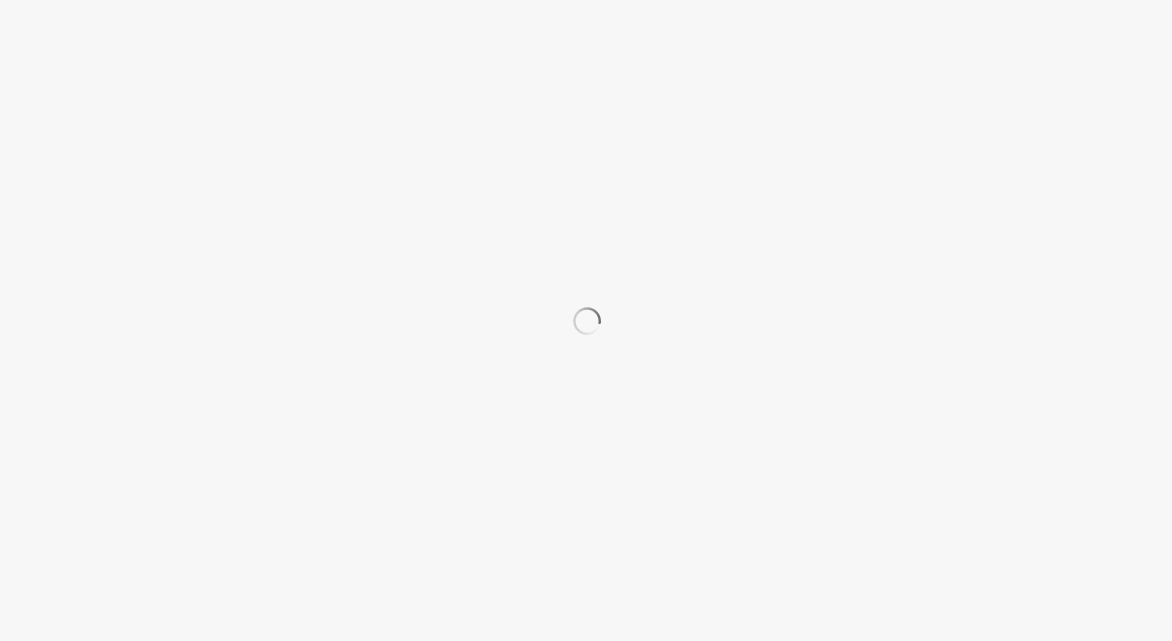


- 什么是实验下线推全

产品层面已经决定下线或者推全对某个模块的实验功能了，在代码中，还残留的代码分支，不易于理解代码，需要结合相关信息才能推测目前的分支走向，这时候研发需要从编码上进行清理，以保证代码逻辑不腐化。

整体设计

既然有了目标和思路，那第一个想法是首先要把这个AB实验下线推全的工作流的轮子给造出来，麻雀虽小，五脏俱全。说干就干，为此，我们完整分析了基于AB实验下线推全生码的工作流所需要的依赖，那基于业务场景需求，我们建了计划构建一套以**A2A协议到AG-UI协议转换为枢纽，结合业务知识动态提示词为核心的的AI智能生码**系统：



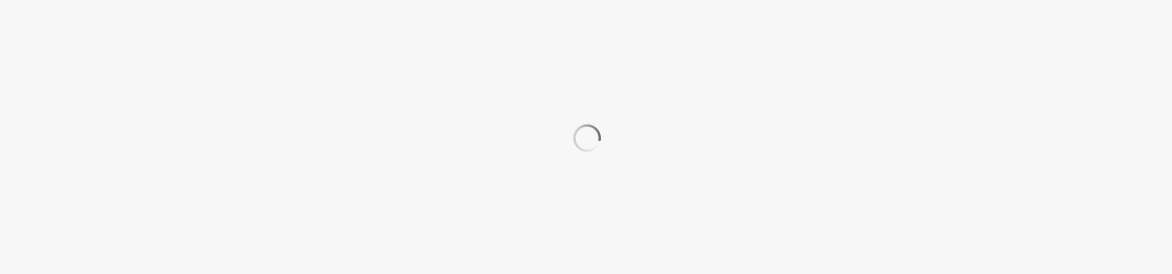
交易生码整体架构

该系统采用分层架构设计，确保系统的可扩展性和稳定性，基于此架构，我们目前需实现 AB 实验下线推全 的完整自动化生码流程。系统的核心运行流程包括：

- **前端交互层：**工作流配置获取、WebSocket连接建立、AG-UI协议传输，实现生成式UI动态展示；
- **中间服务层：**IDE空间管理（HTTP加密接口）、AB实验平台集成（HSF接口查询）；
- **工具集成层：**Aone变更和Code分支创建（MCP工具集成）、A2A协议配置传输（事件处理机制）；
- **数据回写层：**实验治理平台集成（HSF回调）。

模块编排

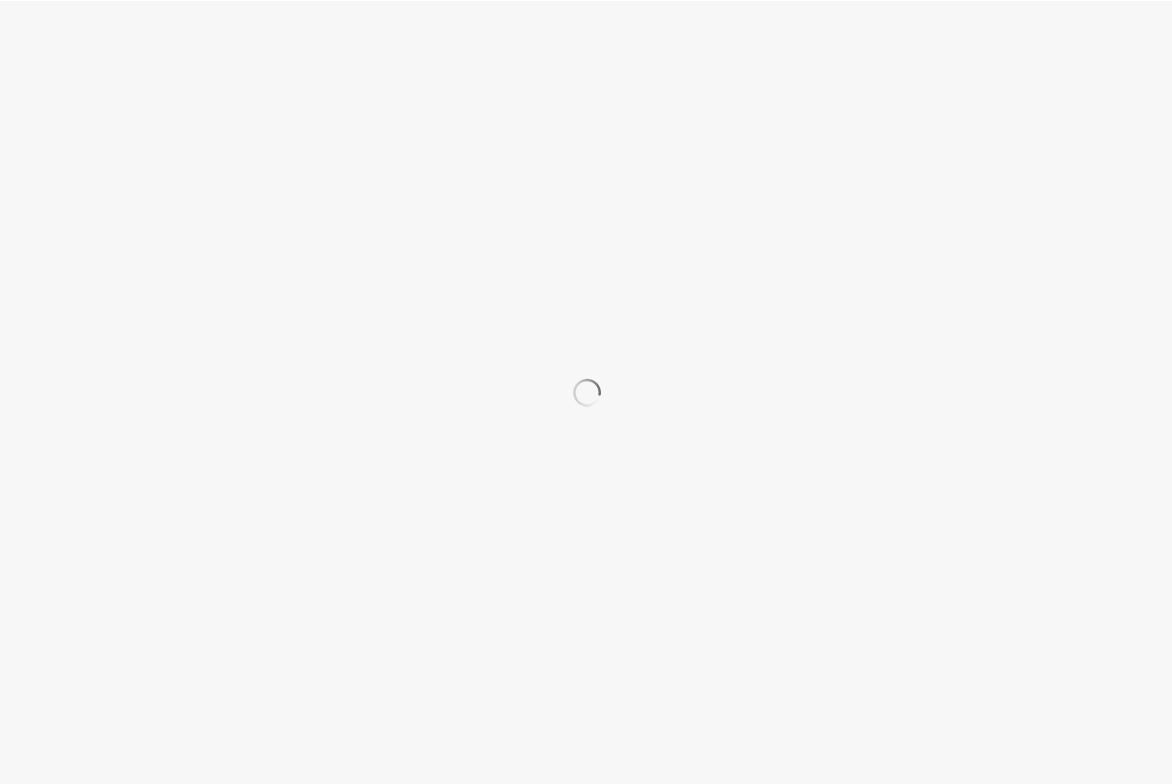
要实现相关工作流的运行，需将整体架构拆分为三个关键模块：A2A协议转换、AG-UI协议通信和Workflow编排引擎模块。



其中A2A模块承担AI代理通信的桥梁职责，实现了流式响应处理和JSON-RPC消息协议的标准化封装。AG-UI服务通过WebSocketService与后端建立实时通信链路，结合前端Next.js框架，实现即时生成式UI的动态渲染效果。Workflow编排引擎负责协调MCP工具链、Diamond配置中心、内部平台服务，通过TaskManager统一管理任务状态，实现工作流的可视化运行监控。

- 运行流程

典型的工作流执行遵循以下标准化流程：



交易生码工作流标准流程

1. **配置初始化阶段：**前端应用调用GET /complete接口获取工作流配置信息
2. **工作流启动阶段：**前端发送POST /start-workflow请求启动工作流，携带实验业务参数
3. **AI代理执行阶段：**Tagent执行核心AI代理任务，包括数据处理和业务逻辑执行
    - 通过POST /a2a向A2A服务发送策略增强和prompt任务请求
    - 调用MCP工具执行具体操作（createBranch、searchCode等标准工具调用）
    - 与内部AB实验平台进行数据交互，获取实验配置信息
    - 向TCS系统发起合并请求，完成代码集成
4. **状态同步阶段：**Tagent通过SSE流式推送向前端实时发送状态更新事件
5. **用户界面更新阶段：**前端通过WebSocket推送机制向用户展示实时进度更新

- 定义工作流

在工作流体系建设过程中，为了增强后续能力**扩展性**和整体系统**开放性**，我们对核心流程进行了**深度抽象和标准化固化**，成功沉淀了一套AB实验下线与Switch开关切流的标准化工作流配置体系。

AB下线工作流

```yaml
# 工作流元数据配置
metadata:
  name: "AB实验下线工作流"
  version: "2.1.0"
  description: "AB实验下线和代码注释工作流"
  totalSteps: 5

# 步骤配置（前端显示用）
steps:
  - id: 1
    title: "创建IDE空间"
    description: "后续的代码操作基于此IDE空间"
    icon: "monitor"
  - id: 2
    title: "获取AB实验"
    description: "获取AB实验的详细信息"
    icon: "database"
  - id: 3
    title: "创建变更"
    description: "创建Aone变更请求"
    icon: "git-branch"
  - id: 4
```

```
    title: "代码修改"
    description: "统一处理AB实验相关的所有代码修改"
    icon: "code"
  - id: 5
    title: "创建代码评审"
    description: "创建CR，并回调TCS平台写入相关信息"
    icon: "upload"
# 卡片生成配置
cardGeneration:
  rules:
    # 步骤1：创建IDE空间
    step1:
      type: "link"
      title: "IDE工作空间"
      template: "{workspaceUrl}"
      action: "open"
    # 步骤2：获取AB实验信息
    step2:
      type: "cards"
      title: "AB实验信息"
      template: "**实验名称**：{expName}\n**目标分组**：{expGroup}\n"
      action: "copy"
    # 步骤3：创建变更和分支
    step3:
      type: "text"
      title: "分支信息"
      template: "分支：{branchName}"
      action: "copy"
    # 步骤4：代码修改 - 工作流信息一览格式
    step4:
      type: "cards"
      title: "大模型修改总结"
      template: "{desc}"
      action: "copy"
      autoCollapse: true  # 第四步完成后自动收起
    # 步骤5：创建代码评审
    step5:
      type: "link"
      title: "代码合并请求"
      icon: "upload"
      action: "open"
```

Switch治理工作流

```
# Trade Agent Switch开关治理工作流配置文件
```

```yaml
# 版本: 1.0.0
# 用于统一管理Switch开关治理工作流步骤和卡片生成等

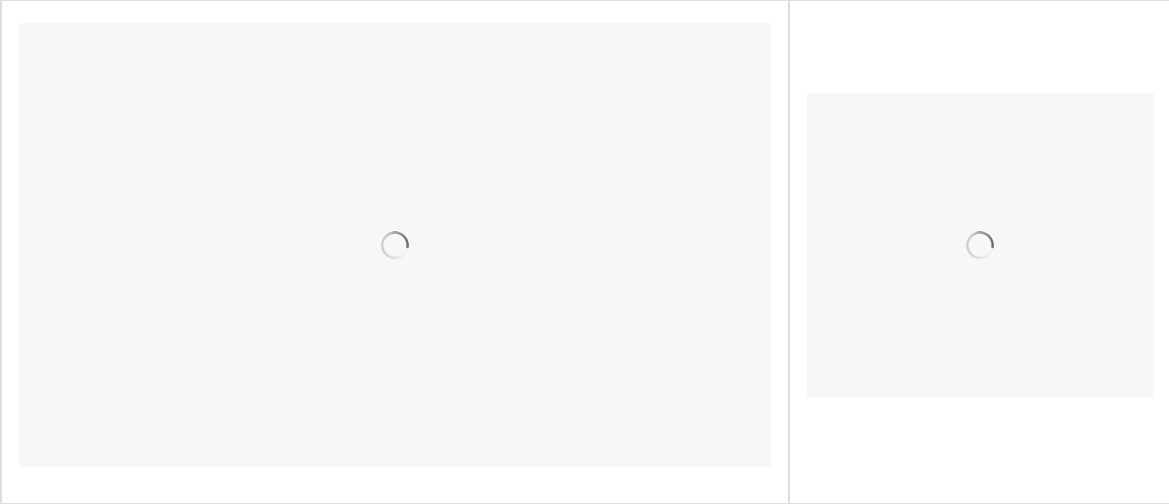
# 工作流元数据配置
metadata:
  name: "Switch开关治理工作流"
  version: "1.0.0"
  description: "智能开关治理工作流，支持开关状态管理和代码自动化修改"
  totalSteps: 5  # 步骤1-5：IDE空间、获取开关信息、创建变更和分支、代码修改、创
  author: "Trade Agent Team"
  lastModified: "2025-01-18"
# 步骤配置（前端显示用）
steps:
  - id: 1
    title: "获取开关信息"
    shortTitle: "开关信息"
    description: "获取Switch开关的代码定义和引用仓库"
    icon: "toggle-left"
    category: "experiment"
  - id: 2
    title: "创建IDE空间"
    shortTitle: "IDE空间"
    description: "后续的代码操作基于此IDE空间"
    icon: "monitor"
    category: "setup"
  - id: 3
    title: "创建变更"
    shortTitle: "变更分支"
    description: "创建Aone变更请求"
    icon: "git-branch"
    category: "branch"
  - id: 4
    title: "代码修改"
    shortTitle: "代码修改"
    description: "AI自主修改代码并提交，耗时3到5分钟"
    icon: "code"
    category: "coding"
  - id: 5
    title: "创建代码评审"
    shortTitle: "代码评审"
    description: "创建CR，并回调Switch平台写入信息"
    icon: "upload"
    category: "review"
# 卡片生成配置
cardGeneration:
  rules:
```

```yaml
    # 步骤1: 获取开关信息
    step1:
      type: "cards"
      title: "Switch开关信息"
      template: "**开关名称**: {switchName}\n**应用名称**: {appName}\n**操
      icon: "toggle-left"
      action: "copy"
      category: "experiment"
    # 步骤2: 创建IDE空间
    step2:
      type: "link"
      title: "IDE工作空间"
      template: "{workspaceUrl}"
      icon: "monitor"
      action: "open"
      category: "setup"
    # 步骤3: 创建变更和分支
    step3:
      type: "text"
      title: "分支信息"
      template: "分支: {branchName}"
      icon: "git-branch"
      action: "copy"
      category: "branch"
    # 步骤4: 代码修改 - 工作流信息一览格式
    step4:
      type: "cards"
      title: "工作流信息一览"
      cardTitle: "开关治理修改总结"
      template: "{desc}"
      icon: "code"
      action: "copy"
      category: "coding"
      autoCollapse: true  # 第四步完成后自动收起
    # 步骤5: 创建代码评审
    step5:
      type: "link"
      title: "代码合并请求"
      template: "{crUrl}"
      icon: "upload"
      action: "open"
      category: "review"
# 参数配置
parameters:
  # 必需参数
  required: ["switchName", "appName", "id", "taskId"]
```

```
94    # 默认参数
95    defaults:
96      branchPrefix: "ai_switch_"
97      template: "devstudio-remote-agent"
98      callSystem: "switch"
```

提示词设计

而对于我们一休实验自动下线，提示词的设计上有啥思路呢？自己也是一个业务开发，对一休ab使用的比较多，也就有一些理解。deepseek的深度思考不是可以对提示词优化嘛，因此我先写了一份我想做的事情，让ai帮我生成一份非常好的提示词。再仔细思考下，还需要加上（推全的组 + 一休实验的key + 一休实验的数据 + 提示词指引）这些动态信息，agent才能帮我准确实现一个实验下线了，下图的执行流程，和人类实现实验下线类似，将其抽象为提示词给agent理解：

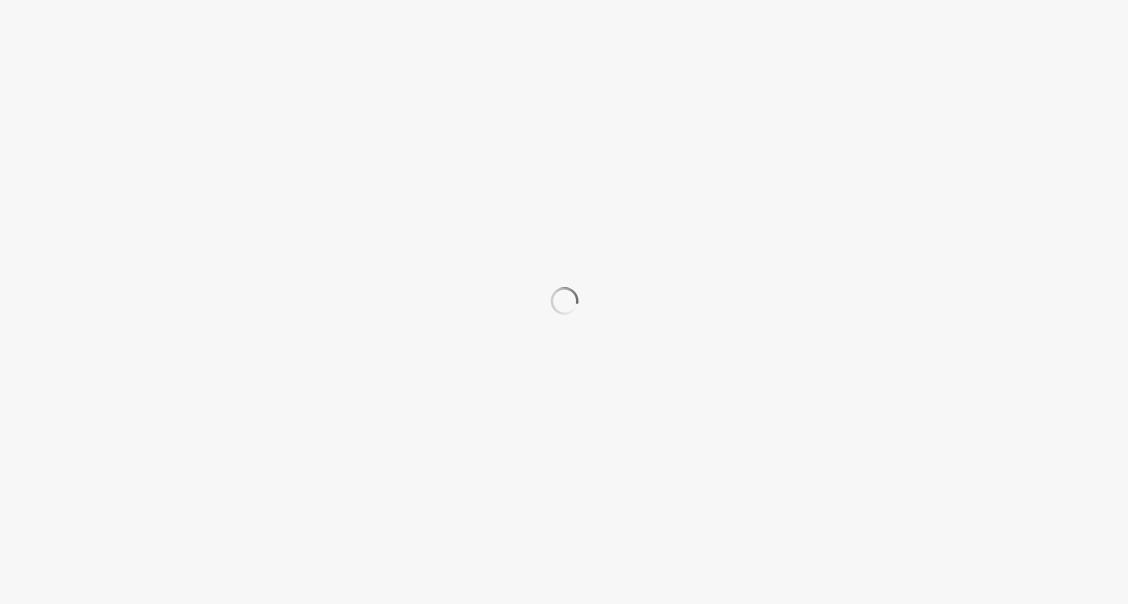


- 什么是好的提示词

如何写好提示词呢？参考了业界对于好的提示的**定义标准和规范**

Openai	moonshot
1. 撰写清晰的指令 2. 提供参考文本 3. 把复杂的任务拆分成简单的子任务 4. 给模型更多时间“思考” 5. 运用外部工具 6. 系统地对变更进行测试	1. 编写清晰的说明 2. 在请求中包含更多细节可以获得更相关的回答 3. 在请求中要求模型扮演一个角色可以获得更准确的输出 4. 在请求中使用分隔符来明确指出输入的不同部分 5. 明确完成任务所需的步骤 6. 向模型提供输出示例

经过多版的思考迭代，提示词内容框架拆分成**全局Rule**和**执行时提示词**， 设计如下：



可以看到“一休的实验数据”是放置在提示词上下文，其实也可以提供MCP的工具由agent自己决定调用。不过一休实验数据调用是一步很明确的动作，直接放置上下文中也是一种方式。

GlobalRule

```
globalRule: |
  Always respond in Chinese-simplified，你是相关功能下线java专家，这次你将负
  以下是你后续工作的一些指导方法
  【执行模式：严格模式 | 温度：0.0 | 禁止变化：每次执行结果必须完全一致】，保证每次
  [4.什么是AB实验]
    1. 举一个例子
       实验是由产品创建的
       以订单详情页样式实验作为示例，其作用是控制订单详情页样式，订单详情有两种样式:
       因此该实验配置了两个变量，pageStyle=[oldStyle,newStyle],recommendAlg
       总共配置了4个实验分组，分别是：
       旧样式+ctr算法:variations={"pageStyle":"oldStyle","recommendAlgo"
       旧样式+cvr算法:variations={"pageStyle":"oldStyle","recommendAlgo"
       新样式+ctr算法:variations={"newStyle":"oldStyle","recommendAlgo":
       新样式+cvr算法:variations={"newStyle":"oldStyle","recommendAlgo":
       同时有一个特殊实验分组概念：
       全局空桶分组:variations={},当命中该分组，pageStyle、recommendAlgo都获
       流量的划分是，全局空桶分走n%(n <= 2%)的流量，剩余所有流量由产品指定百分比分
    2. 代码中如何获取实验结果
       一般来说是通过一些实验工具类，传入用户Id、实验key，获取实验结果
       如：
       情形1:
       //先获取实验结果对象
```

```
        AbTestResult abTestResult = CartABTestManager.getInstance().getA
        String pageStyle = (String)abTestResult.getVariations().get("pag
        String recommendAlgo = (String)abTestResult.getVariations().get(
        情形2:
        //工具类帮我们处理，获取variations中某个key对应的value
        String pageStyle = AbHelper.getVariation(userId, ChinaMainSiteSv
        String recommendAlgo = AbHelper.getVariation(userId, ChinaMainSi
    3. 如何使用获得的变量
        如，开发人员是知道变量pageStyle、recommendAlgo对应有哪些枚举值：
        if (pageStyle.equals("newStyle")) {
          // 新样式处理
        } else if (pageStyle.equals("oldStyle")) {
          // 旧样式处理
          //这种情形下，命中全局空桶分组，在业务上隐含的含义就是使用旧样式
        }
        if(recommendAlgo.equals("ctrAlgo")) {
          // ctr算法处理
        } else if (recommendAlgo.equals("cvrAlgo")) {
          // cvr算法处理
        } else{
          //推荐模块不出
          //这种情形下，命中全局空桶分组，在业务上隐含的含义就是不展示推荐模块
        }
[5.任务指令理解]
    代码中对于实验的处理，存在上述所说的逻辑分支，我们的任务就是将指定实验的分支逻辑
    就会有两种操作概念
    推全(openAll):将所有流量（包含全局空桶的100%的流量）推全到某个指定的实验分组
    下线(shutdown):类似全局空桶分组的逻辑，将实验相关逻辑下线，不再调用实验平台，
[6.执行必须遵守的一些规范]
    1. 准确识别和任务相关的内容
        1. 例子1
            实验可能会通过一些实验工具类去获得实验分组中的某个key
            AbTestUtil.hit(expKey, buyerId, "hit");
            AbTestUtil是实验的工具类，expKey是实验的key，buyerId是用户id，"hit"
            实验工具类会通过不同的expKey去调用，不要通过实验工具类去扩散到其他实验，实
        2. 例子2
            ABTestResult result = tradeFlowClient.flow(userId, experimentK
            experimentKey使我们传入的实验key（即expKey），不要通过tradeFlowClien
        3. 例子3
            boolean greyFlag = RxHelper.abTest(buyerId);
            这个没有传入实验key，说明不是一个通用的实验工具类，专门只给一个实验使用，
    2. 通用实验流程处理的限制，举一个例子
        @AppSwitch(des = "", level = Level.p3)
        public static Map<String, String> scene2AbCode = new HashMap<>()
        //confirmGood场景对应的实验是"AB_2312312312"
        static{
```

```
            scene2AbCode.put("confirmGood", "AB_2312312312");
        }
        //一个通用实验方法，根据场景scene获得对应的实验expKey，然后做实验
        public String getAbCode(String scene, long userId) {
          String abCode = scene2AbCode.get(scene);
          if(StringUtils.isEmpty(abCode)){
            return null
          }
          return AbUtils.ab(userId, abCode, "key");
        }
        即便我们想要推全实验AB_2312312312，并且scene2AbCode确实也只有"confirmG
        因为该方法目的就是，为了以后可以在scene2AbCode加入其他场景去做实验，即便现
        这种场景我提供一个解法，当然你也可以自己实现优雅点的方式
        Map<String, String> scene2Variation = new HashMap();
        假设推全的实验分组，获取的该变量对应的值为"true"
        scene2Variation.put("confirmGood", "true");
        public String getAbCode(String scene, long userId) {
          //这样会优雅点，针对不桶的scene都可以推全
          if(StringUtils.isNotBlank(scene2Variation.get(scene))){ return
          String abCode = scene2AbCode.get(scene);
          return AbUtils.ab(userId, abCode, "key");
        }
      3. 禁止改动的方法
        举例：
        class GrayConfig{
            private String expKey;
            public String ab(long userId){
                return AbUtils.ab(userId, expKey, "key");
            }
        }
        判断后，
        GrayConfig是一个关于expKey实现的通用配置类，ab方法适合GrayConfig的实例
        因此我们不能去改ab方法的通用实现。
        因此在对方法内的代码进行变更时，判断方法是否是实例维度的，或者说是通用，共有
```

ContextPromote

```
step1: |
  使用深度思考模式执行
  本步骤将统一处理AB实验相关的所有代码修改，包括实验定义注释、调用逻辑注释、依赖清理
  **执行参数**
    - expKey: {expKey}
    - experimentGroups: {expGroup}
      type: array
        items:
```

```
          id: 实验分组id，唯一标识一个分组
          name: 实验分组名称，代码中感知不到
          variations:
              type: array
                  items:
                      key: 实验分组中的某个key
                      value: 实验分组中的某个key对应的value
      解释：实验分组，一个实验所有的实验分组信息
  - 操作类型：{closeAction}
      enum: [ "shutdown", "openAll" ]
      解释：shutdown表示下线实验，openAll表示推全实验（推全到某个实验分组）
  - bucketId: {bucketId}
      解释：（需要推全的目标实验分组ID（操作类型=openAll时必填），表明代码逻辑中只
第一阶段：分支切换
  注意如果git命令唤起terminal如果失败，要持续尝试
  - 如果暂存区有未提交的代码，直接删除即可
  - 执行 `git fetch origin {branchName} --depth=10 --quiet` 仅拉取指定分
  - 执行git命令以切换到指定分支，参考 `git checkout {branchName} --quiet`
  - 注意：git命令的详细输出不要加入到LLM分析上下文中，仅关注命令执行结果状态
  - 如果切换分支执行失败，终止任务
第二阶段：实验逻辑分析与执行
  1. 找到该实验{expKey}在仓库中的引用，可以使用搜索功能完全匹配；如果找不到就不需
  2. 对上一步中找到的引用称为expKey定义的位置，可能为常量，如果定义没有被引用，说
  3. 对于上述找到的expKey引用，向上广度扩散追溯3层，了解实验的逻辑处理，要准确识
  4. 根据"操作类型"类型执行对应的任务操作,操作时，对一些方法进行更改时，要根据gl
第三阶段：代码逻辑优化
  1. 上述流程执行完成后，我们已经完成该实验相关的代码修改，此时我们要注意架构设计，
  2. 实现代码inline
  3. 清理不再使用的代码依赖、函数和变量
  4. 可以删除的代码
    有些是用于测试，如白名单、黑名单、enableThisAbAll（强制开启），这些逻辑在实
    if(whiteUserIds.contains(userId)){ return variations}
    if(blackUserId.contains(userId)){return null}
    if(Setting.enableThisAbAll){return variations}
    有些是业务逻辑限制，如客户端版本限制、tabCode限制，这些和实验无关，不能改动。
      if(客户端版本 <= '10.20'){return null}
      if(tabCode != 'confirmGood'){return null}
第四阶段：代码注释修改
1. 按要求注释实验相关代码
2. 检查应用名称{appName}是否包含product字段
  - 如果是产品包，pom.xml中找到版本号并升级为下一个版本的-{今天日期}-SNAPSHOT版
3. 检查并修复语法错误
4. 暂存和提交代码
  - 执行git命令提交代码修改（使用--quiet参数减少输出）
  - 执行git命令推送代码到远程分支，参考：`git push origin HEAD --quiet`
  所有子步骤完成后，必须按照以下格式输出结构化数据：
```

```
[DATA_OUTPUT]
{
  "stepNumber": 4,
  "desc": "[格式化输出本次修改的大模型给出的详细总结，需包含实验完整信息，如
  "status": "completed"
}
[/DATA_OUTPUT]
然后输出：已完成统一代码修改工作 | [STEP_COMPLETE] Step4 执行完成
```



轮子上线

轮子内部

通过策略模式分离抽象出来AB推全下线和Switch治理工作流：

- WorkflowStrategy接口定义通用工作流规范
- AbExperimentWorkflowStrategy和 SwitchWorkflowStrategy实现具体策略
- WorkflowStrategyFactory根据工作流类型动态选择策略



轮子外观

新造的轮子由于采用的全新的技术栈，具有以下特点：

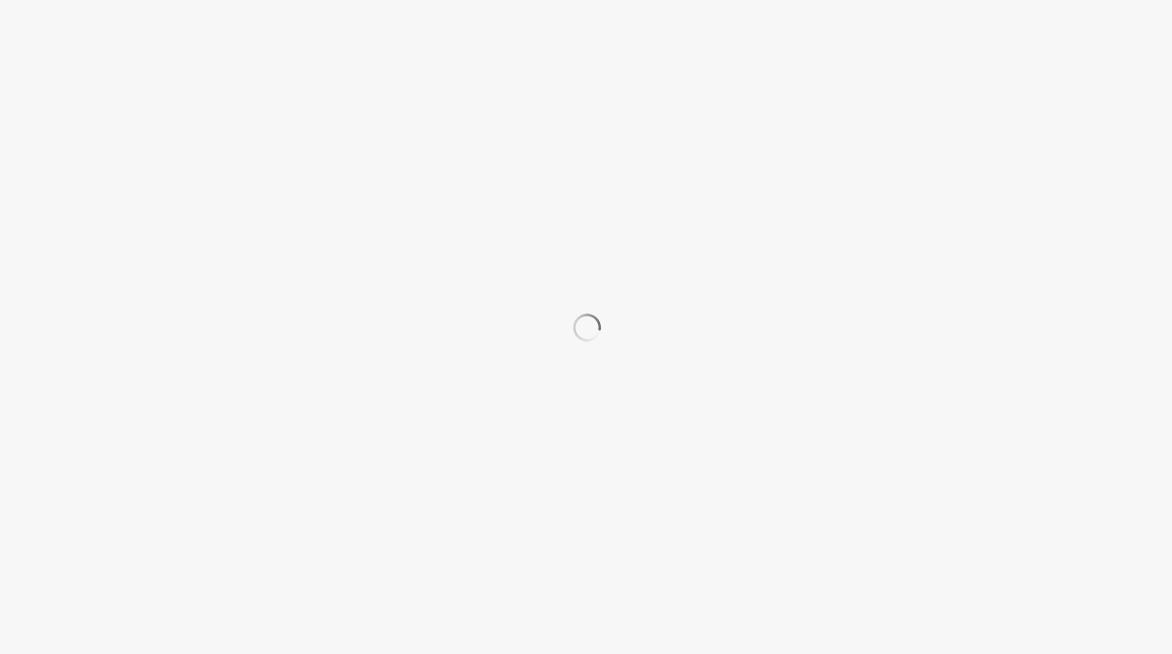
💬 实时代理聊天与流信息

🔄 长连接双向状态同步

🧩 生成式 UI 和 结构化消息

🧠 实时操作日志及上下文信息展示

👷 人机协作，可中途介入干预



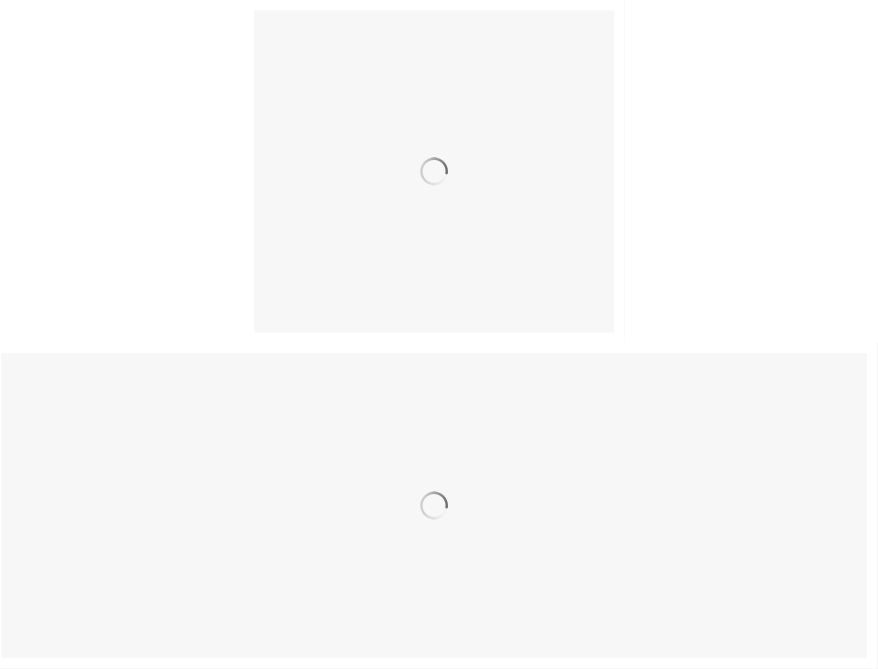
轮子试跑

通过 **DxxxAgent 提供的远程 CodeAgent** 能力，结合 **AG-UI + A2A + MCP**协议，最终配合我们通过各种接口获取的动态上下文，整体流程终于跑通了：

但从技术的成长来看，“Demo上线”只是项目的开始的第一步，证明使用AICoding工作流的能力完成AB实验的自动推全和下线其最小闭环路径是可行的，但是用户如何鉴权 ？ 长连接状态如何保持 ？ 断线状态如何恢复 ？ 实时的日志如何打印 ？ 怎么和实验治理平台进行协作还需要一步步优化。

可惜轮子的成绩不太行

优化功能需求的过程中，还会遇到工程实践上越不过去的坎，那就是准确率的问题！通过整体生码流程的运行分析，我们识别出单个任务工作流执行结果准确性的关键制约因素：大模型能否从现有MCP工具、接口集成及知识库中获取充分的上下文信息，以实现精确的代码修改。



上下文信息的完整性和语义的准确直接决定了生码质量，项目起初，使用原版静态提示词的成功率不到50%，很多时候完全依赖模型去猜，如何获取动态数据 ？ 如何纠正模型的行为 ？如何提升生码的确定性 ？这成为当前阶段需要重点突破的技术瓶颈。



提升准确率

做成一件事情，有很多因素，“坚持”，是成本最低的一种。

提示词的优化

在模型实际执行过程中，最关键的工作是对修改结果进行客观评估。在批量执行过程中，结合模型执行的具体表现，我们抽象出模型思考过程中的异常思维模式并进行剪枝优化，确保模型生成的代码不仅技术准确，同时能够更好地契合实际业务场景需求。

AB实验工作流提示词

```
# 全局规则 - 适用于所有步骤的基础配置
globalRule: |
  Always respond in Chinese-simplified，你是相关功能下线java专家，这次你将负
  以下是你后续工作的一些指导方法
  【执行模式：严格模式 | 温度：0.0 | 禁止变化：每次执行结果必须完全一致】，保证每次
  【进度控制规则】：
  · 在每个步骤最终完成时，必须在结论最后输出：`[STEP_COMPLETE] Step{步骤编号}
```

```
    · 严格按照预定义的操作序列执行，不得跳过或重排序
    · 重要：切勿在开始时输出开始执行信息，只在每个操作完成时在结论最后输出完成信息！
    【数据返回格式规范】：
    · 每个步骤完成后必须按照指定的JSON格式输出结构化数据
    · JSON数据必须包含在 `[DATA_OUTPUT]` 和 `[/DATA_OUTPUT]` 标签之间
    · JSON格式必须严格遵循，不得有额外的文字或格式
    · 注释修改规范：
     ```java
     // 大模型自动注释代码，确认后可手动删除
     /*
     需要注释的代码
     */
     ```
    ✅ 只能通过注释完成相关的修改，只能有注释相关的修改（添加注释块）
    ✅ 注释格式必须完全符合规范
    ✅ 仅校验被修改的代码是否存在逻辑问题，复杂条件中保持逻辑语义一致，优化整体代码书
    ✅ 举例：若出现以下类似逻辑，可一并注释 - if(true) 则一并注释 if(true)
    - if(false) 则注释if(false) 及 逻辑判断中的代码并注释
    ✅ 出现类似多重逻辑，只能注释其中ab返回condition2的逻辑，需保留其他逻辑condit
    ❌ 不允许任何新增的import语句，不允许任何新增的代码逻辑
    ❌ 不允许任何直接删除操作（除了被注释的内容）
    ❌ 不允许任何新增import语句
    ❌ 缩小注释的范围，避免在已有注释上注释，导致注释失效的情况发生：
    ❌ 修改后的代码，需要确认逻辑正确，不允许出现类似恒正确或者恒错误的判断
    ❌ 如出现以下逻辑，应该直接将该函数注释，并在调用处返回false
step4: |
    步骤4：代码修改 - 耗时约5-10分钟
    本步骤将统一处理AB实验相关的所有代码修改，包括实验定义注释、调用逻辑注释、依赖清理
    - 任务：
    我们的开发工作中，会通过实验平台的sdk，请求expKey去获得该用户的实验结果，用户可能
    对于一个实验来说，产品会配置n个实验变量，对每个实验变量（key）来说，key是固定的
    简单的实验，一般会配置一个实验变量，复杂的实验可能会控制多个模块的展示，那就会有多
    一个实验会有多个实验分组，产品一般会将实验变量的值去进行组合。
    从服务端的角度，会通过实验平台的sdk，请求expKey去获得该用户的实验结果，返回用户命
    服务端会通过variations.get(key)获取value，去做相应的逻辑处理。服务端、代码中是
    每个实验有多个实验分组，每个实验分组有对应的variations，说明：
      - 对每个实验来说，variations中的key是固定的，代码中会使用key去获得该实验分组
      - 比如对实验A来说，key有"key1","key2"，开发通过variations.get("key1")去
      - 因此代码中只感知key对应value的不同，而不感知实验分组的存在
    全部信息：
    - expKey: {expName}
    - experimentGroups: {expGroup}
      type: array
        items:
          id: 实验分组id，唯一标识一个分组，代码中感知不到
          name: 实验分组名称，代码中感知不到
```

```
        variations: 实验变量的key-value，是一个map，代码中命中改实验分组时，可
        description: 实验分组描述
    解释：实验分组，一个实验所有的实验分组信息
- 操作类型：{closeAction}
    enum: [ "shutdown", "openAll" ]
    解释：shutdown表示下线实验，openAll表示推全实验（推全到某个实验分组）
- bucketId：{bucketId}
    解释：（目标实验分组ID（操作类型=openAll时必填），表明代码逻辑中只保留该实验分
- variations：{bucketVariations}
    解释：这个只在"操作类型" = openAll 的时候有意义，是指定bucketId（实验分组）
- 代码仓库：{codeRepo}
- 分支名称：{branchName}
你需要按照以下步骤执行：
**第一阶段：实验逻辑分析**
1. 理解实验配置
    - 每个实验有多个实验分组，每个实验分组有对应的variations
    - 对每个实验来说，variations中的key是固定的，代码中会使用key去获得该实验分组
    - 比如对实验A来说，key有"key1","key2"，开发通过variations.get("key1")去
    - 代码中只感知key对应value的不同，而不感知实验分组的存在
    - 根据"操作类型"确定操作：openAll(推全)或shutdown(下线)
2. 任务类型分析
    - when: 操作类型 == "openAll": 保留该实验分组分支逻辑，将其余实验组的分支逻
    - when: 操作类型 == "shutdown": 将实验相关逻辑下线，不再调用实验平台，var
3. 代码引用扫描
    - 搜索所有{expKey}的字符串字面量引用
    - 向上追溯3层，了解实验的逻辑处理
    - 识别实验相关的常量、方法、类等
    - 保证注释后代码逻辑完整性
    - 确认所有修改都符合注释规范
        - 对注释后的代码进行逻辑优化，确保语义一致性
    - 处理复杂条件中的逻辑，保持业务逻辑完整性
    - 检查逻辑完整性，避免引入运行时异常
    - 通用模版类型的实验处理
    - 代码扩散限制
      实验范围与 expKey 关联，不能通过实验工具类扩散到其他实验
    - 实验灰度相关的代码处理
      有些是用于测试，如白名单、黑名单、enableThisAbAll（强制开启），这些逻辑在实
      有些是业务逻辑限制，如客户端版本限制、tabCode限制，这些和实验无关，不能改动
**第二阶段：代码注释修改**
1. 执行git命令：`git fetch origin` 获取远程分支信息
    - 执行git命令：切换到指定分支，参考 `git checkout {branchName}`
2. 按要求注释实验相关代码
3. 按要求推全分组或下线实验
4. 清理不再使用的代码依赖、函数和变量
5. 实现代码inline，简化方法栈调用
6. 检查应用名称{appName}是否包含product字段
```

```
  - 如果是产品包，pom.xml中找到版本号并升级为下一个版本的-{今天日期}-SNAPSHOT
7. 检查并修复语法错误
8. 暂存和提交代码
  - 执行git命令提交代码修改
  - 执行git命令推送代码到远程分支，参考：`git push origin HEAD`
  所有子步骤完成后，必须按照以下格式输出结构化数据：
  [DATA_OUTPUT]
  {
    "stepNumber": 4,
    "desc": "[格式化输出本次修改的大模型给出的详细总结，需包含实验完整信息，如
    "status": "completed"
  }
  [/DATA_OUTPUT]
  然后输出：已完成统一代码修改工作 | [STEP_COMPLETE] Step4 执行完成
```

Switch治理工作流提示词

```
# 全局规则 - 适用于所有步骤的基础配置
globalRule: |
  开关治理背景：
    开关是通过 @AppSwitch 注解定义的配置项，主要用于：
      ● 业务灰度发布
      ● 新功能开关控制
      ● 动态更新频繁变更的配置
    随着业务迭代，很多开关已废弃，但长期没有人清理，导致代码维护成本上升。
  执行开关治理需要遵守的规则：
    1、修改的开关字符串，需要和用户给定的完全匹配，字符不能多，也不能少，不要修改其
    2、保持代码可读性和原有风格一致
    3、若发现引用无法安全移除，请明确标注风险。
    4、代码重构完成后，你需要再review下所修改的代码，确保代码无冗余且逻辑正确
  执行开关治理需要做的事情：
    1、执行git命令拉取最新分支，并切换到用户提供的获分支：`git fetch origin` &
    2、定位开关定义：在项目中查找该开关的定义，确认其变量名、类型和所属类。
    3、查找所有引用位置：搜索项目中引用该开关的所有 Java 文件，若没有搜索到，则任务
    4、代码重构：针对引用该开关的代码，逐个进行语义分析，理解代码含义，执行内联替换
      bool类型的开关：
        重构前：
        ```java
            public static boolean supportTabFilter(CartFilterTabModel
              /** tab下各个tab是否在灰度范围内是独立，有任何一个tab在灰度范围内
              if (!CartABTestSwitch.openNewCart){
                return false;
              }
              return model.isSupportFilter() && CollectionUtil.isNotEm
```
```

```
        }
```
重构后：
```java
  public static boolean supportTabFilter(CartFilterTabModel mo
    /** tab下各个tab是否在灰度范围内是独立，有任何一个tab在灰度范围内，
    return model.isSupportFilter() && CollectionUtil.isNotEmpt
  }
```
重构说明：

因为openNewCart开关的类型是bool，且值是"true"，取反后，表达式执行值为f

String类型的开关：

重构前：
```java
  if (StringUtils.isNotBlank(itemResult.getPay().getQueryDis
    itemResult.getPay().setQueryDiscountedTitlePrefix(CartIt
  }
```
重构后：
```java
  if (StringUtils.isNotBlank(itemResult.getPay().getQueryDis
    itemResult.getPay().setQueryDiscountedTitlePrefix(CartIt
  }
```
重构说明：

因为defaultDiscountedTitlePrefix的值是"券后价"，直接替换开关值

集合/列表类型的开关：

重构前：
```java
  if (CartABTestSwitch.enableFilterGroupCheckWhiteList.conta
    return true;
  }
```
重构后：
```java
  Set<Long> enableFilterGroupCheckWhiteList = JSON.parseObje
    if (enableFilterGroupCheckWhiteList.contains(MtopContext
    return true;
  }
```
重构说明：

因为enableFilterGroupCheckWhiteList的值是"[123,345]"，类型是Set<L

对象类型的开关：

重构前：
```java
  public static boolean invalidFrontGray() {
```

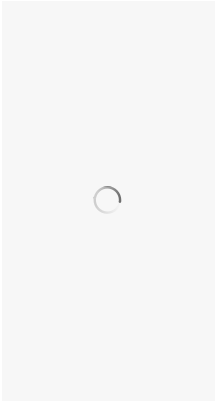
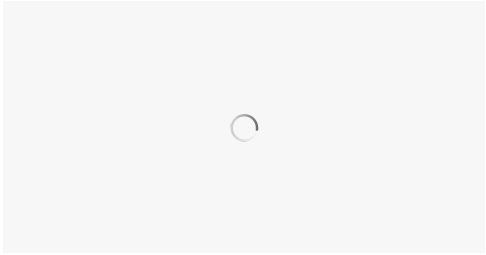
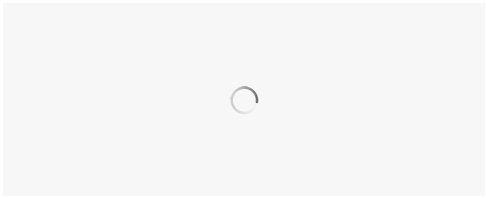
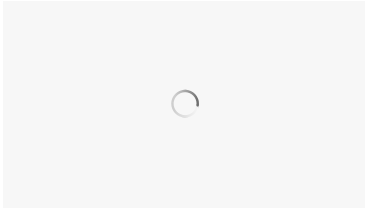
```
74              return CartCustomGrayUtil.isShotCustomCartGray() && Commor
75            }
76            ```
77          重构后：
78          ```java
79            public static boolean invalidFrontGray() {
80              GeneralGraySwitchModel newInvalidFrontGray = JSON.parseObj
81              return CartCustomGrayUtil.isShotCustomCartGray() && Commor
82              return true;
83            }
84          ```
85          重构说明：
86          因为newInvalidFrontGray开关是个对象，因此将开关实际值解析为GeneralGra
87    5、删除开关定义：删除该字段定义及其 @AppSwitch 注解。注意不要删除无关的行。
88    6、检查并修复语法错误
89    7、提交代码：执行git命令提交代码修改，并推送代码到远程分支，参考：`git push or
90    8、总结本次修改总结内容，如果有风险需要用户确认，也一并输出，输出格式如下：
91      [DATA_OUTPUT]
92          本次主要修改了一下文件：
93          1. xxx.java，修改内容：xxx
94          2. xxx.java，修改内容：xxx
95          本次修改有如下风险需要关注：
96          1. xxx
97          2. xxx
98      [/DATA_OUTPUT]
99  step: |
100   我需要治理的开关是{switchName}，开关线上值为{switchValue}，分支名是{branchN
```

通过融合AB实验业务知识、二方平台实验数据、Switch待治理开关关联等多维信息，我们构建了针对性的提示词优化体系，显著提升了模型执行的准确性和稳定性。

工欲善其事必先利其器

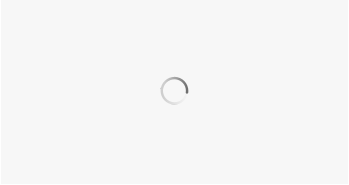
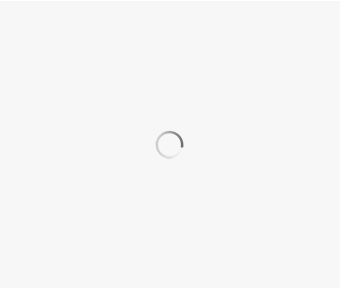
提示词的优化是个逐步演进的过程，一次代码修改经常要花5-10分钟，同时还需要查看它每一步的执行过程。后续同时需要评测其他模型，一个一个实验同步跑，肯定不现实，根据工程的成熟度来说自动化的演进包括三个阶段：

阶段1 单步调试	阶段2 工作流串联	阶段3 批量脚本
提示词 + agent 效率：高	tagent (工程) +提示词+ agent 效率：低	批量脚本任务 效率：中

• 使用CodingAgent原生的Cli界面，可以快速观察执行的过程 	• 工程上基本把所有流程串起来了，但是评测起来等待以及断连还是会比较影响进程同时由于CodeAgent处理是一个完整原子的过程，稍微做一次非常小修改就需要重头开始跑！ • 跑25个实验，等待+汇总+评测基本上就要4个小时，哪吒都看完两遍了！  	• （为什么是中呢，因为写脚本、调试脚本太废人了，脚本不仅仅是要执行，还需要将所有影响准确率相关的信息进行汇总） • 人工成本主要在评测上，评测在30分钟 

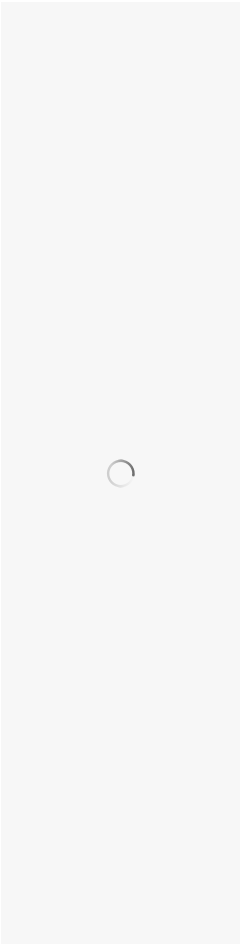
批量脚本的作用

有了批量脚本的执行，我们可以更快的汇总结果，更全面的分析批量执行的过程，更高最终成功率。感谢 @海丫 正因为有了批量脚本，感觉像跑车有了跑道，赛马有了赛场，我们的轮子不用再泡在实验室里了。

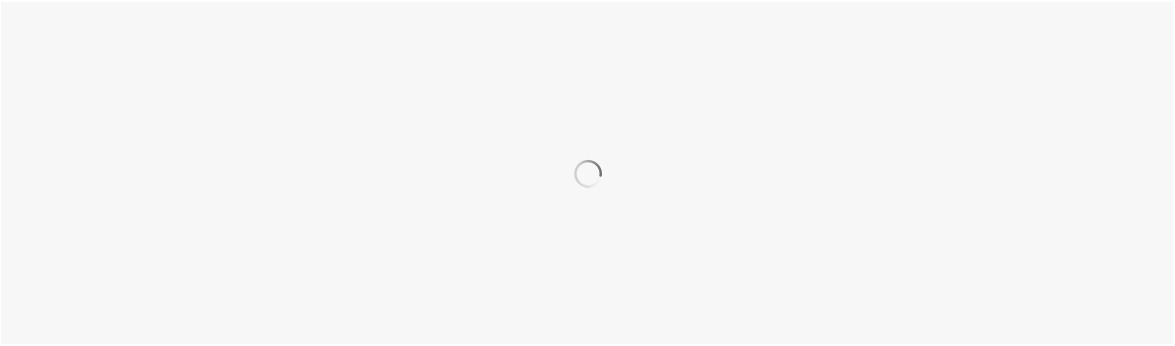
汇总实验结果	记录agent日志	重试机制
• 自动生成表格 • 汇总数据 • 记录执行流程节点	• 断联原因排查 • 模型异常分析 • 查看执行过程	• 初期失败率高 • 减少资源竞争的影响 • 查看每次失败的原因
		

深入分析模型之间的差异

在QwenCoder准确率调优的过程中，我们发现其中有个 逆向钱款去向空白治理 在同样的提示词下，始终无法和Claude4完成类似的修改效果，通过完整分析两个模型的执行日志：以Claude-4和QwenCoder为例进行对比分析，对其过程进行梳理和总结：

流程梳理	过程抽象	结果分析
		Claude-4偏向于提前理解系统架构，进行精确搜索和批量修改； QwenCoder倾向于采用渐进式搜索策略，逐步定位问题，遇到异常后进行重试兜底。
		Claude-4在代码搜索阶段关键字定位更为精确，对结果处理更加严格； QwenCoder处理方式更为宽泛，需要在提示词设计中增加更多约束条件。
		Claude-4在工具使用方面根据修改规模进行提前规划，选择合适的执行方式，主动避免错误发生； QwenCoder更倾向于采用广度优先遍历策略，通过渐进式方法达到最终目标，

基于我们的实践经验，不同大模型对提示词的理解和执行策略存在显著差异。我们发现 Cluade4 更偏向提前理解架构，精确搜索，批量修改，QwenCoder偏向渐进式搜索、逐步定位、遇到错误后再重试兜底。



AI代理与模型的交互可划分为分析阶段和执行阶段两个关键环节。为确保高质量输出，保障模型在分析阶段具备系统性和前瞻性思维，我们提示词设计需要**针对性地阐释实验的业务意义和实际数据背景**，同时消除理解上的歧义。为了提升其在执行阶段更加精准。提示词的设计需要**明确告知模型递归分析的层次深度，以及防止扩散的边界规则**，同时需要为模型提供**明确的修改指导方式**，针对不同模型的处理特点和预期差异进行**提前校正**。



最终评测效果

Cluade4在代码搜索阶段关键字**定位更为精确**，对结果**处理更加严格**。在工具使用方面根据修改的量级提前规划，在验证步骤时能整体考虑项目类型选择合适的执行方式提前避免报错发生。正是因为上述原因才导致最终结果上的差异：

Claude4模型	Qwen3-Coder
最终成功率 92%	基于Claude4的提示词 66% 工程优化后 82%

阶段性成果

交易生码系统V1.0 AB实验自动化治理正式上线 于8月26日正式发布，截止目前，已通过AI生码功能完成68个实验的下线代码生成， 已完成 41 个实验的治理达到待发布状态，AI治理相较于原始手动治理效率提升 70%



反思

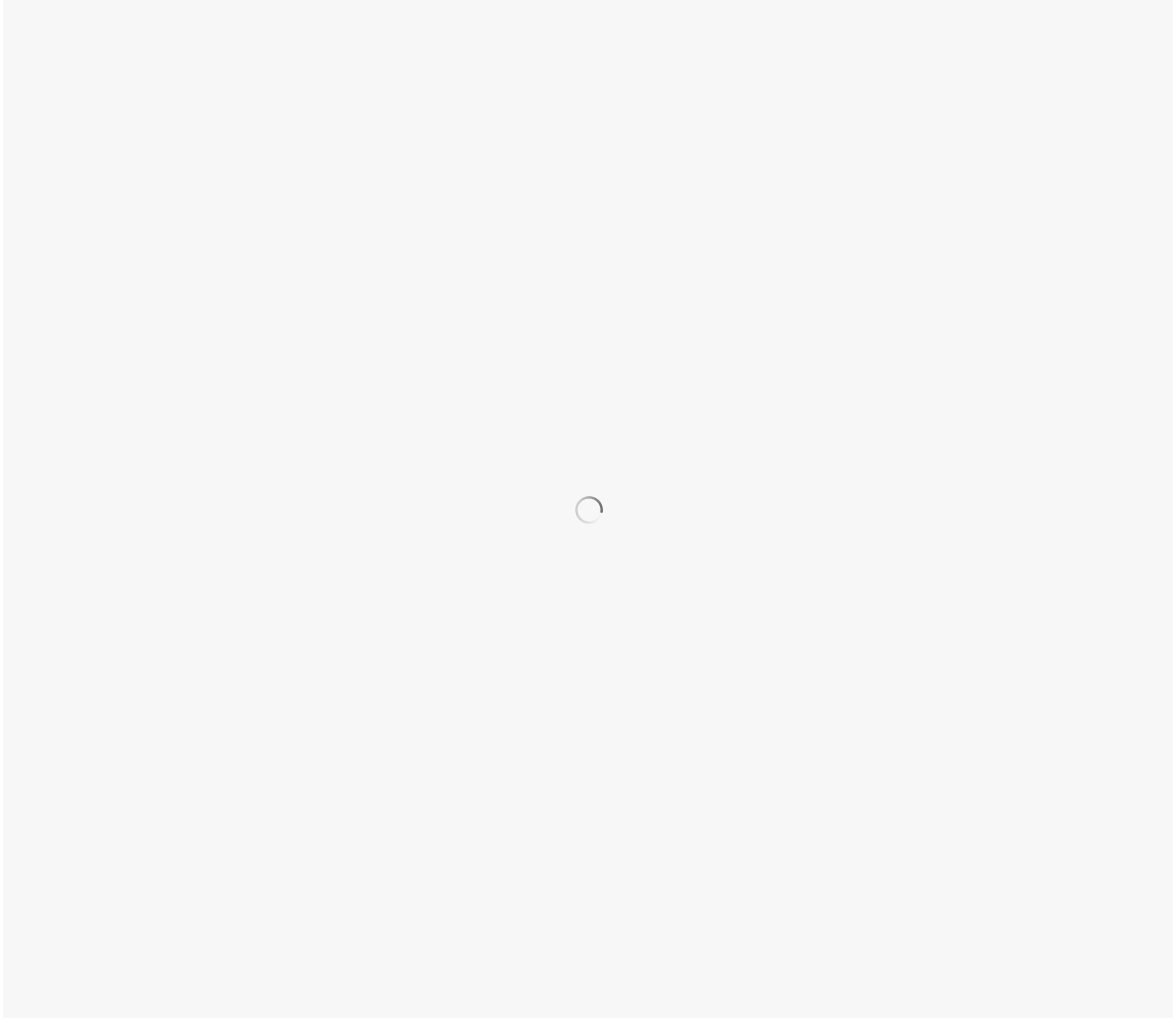
在新的领域大家都是白纸，从0开始，去探索，不要设限，不知不觉的，你思考的问题就会比别人更全面。我在实践有关突破创新的规划时，时常扪心自问：这个方向是否正确？是否真的是未来的趋势？后来我发现，这个问题不对。应该是，你想让这个方向变成未来的趋势吗？没有人会笃定知道这个方向是趋势，但是我想成为让它变成趋势的铺路人。希望有更多有同样想法的人一起沟通进步~



再谈价值

要把事情做到极致，最核心的是要找个好的目标，评判的标准也很简单：当你跟大家说你要做这个事情时，看大家的第一反应是什么？如果是「你怎么做到的」？那么恭喜你找到了一个非常好的目标；如果大家反应是「你为什么要做」，那么你可以再探索探索。

“我从小就知道地球是圆的，但是当我从太空看到地球真的是圆的那一刻，我的感觉才如此深刻。”一位从外太空回来的人这样分享过。哪怕是同样一场演唱会，同样一场球赛，看电视转播和在现场体会是完全不一样的感受。你必须亲临其境，才能获得真实体感。在技术人前进的道路上，看一千遍、一万遍别人的想法，都不如自己去经历一遍。知易行难，祝大家在实践的道路上披荆斩棘、勇闯难关~





团队介绍

本文作者淘苏，来自淘天集团–交易业务技术团队，本团队专注于下单、支付、物流、服务、逆向退款等核心交易链路的技术团队，我们面向亿万用户致力于打造极致流畅、安全稳定的购

物体验，为用户和商家的每一次交易行为保驾护航。同时我们也在持续推进交易平台架构演进，全程保障每一次大促的平稳运行，在流量洪峰的考验下确保交易链路的丝滑体验。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 2.0万